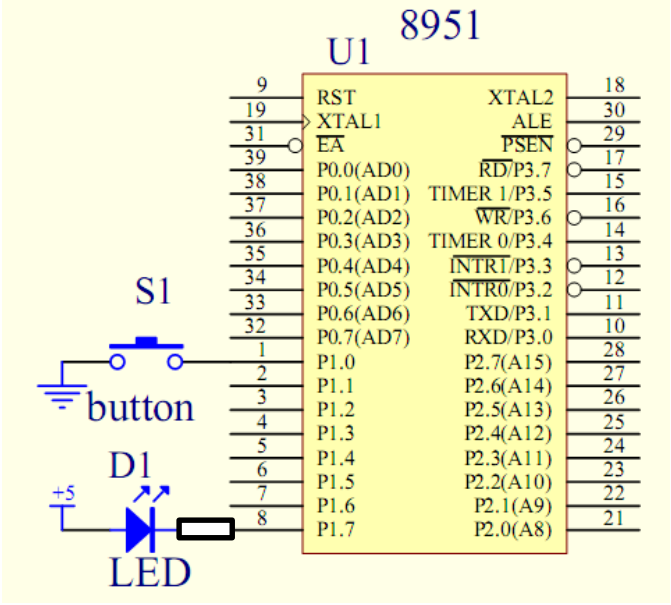
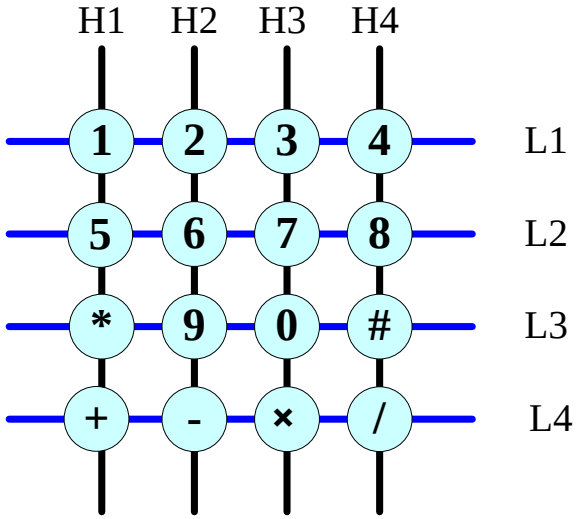
8.3-微机系统键盘接口

分类：

1.独立键盘



2.矩阵键盘



按键（轻触开关）和开关



按键



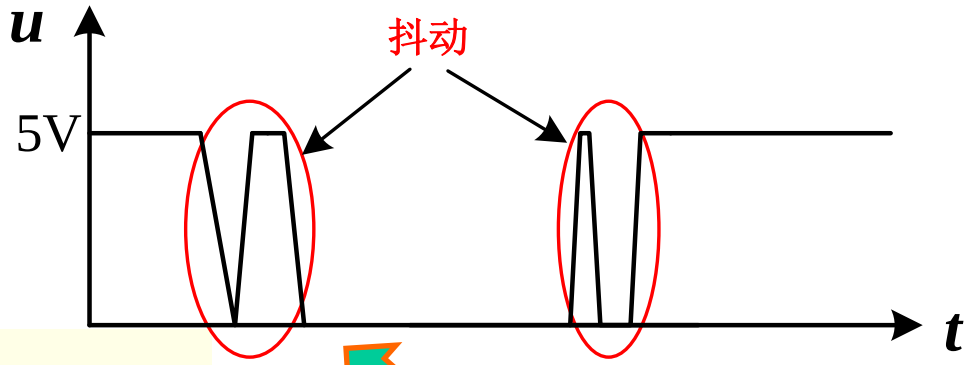


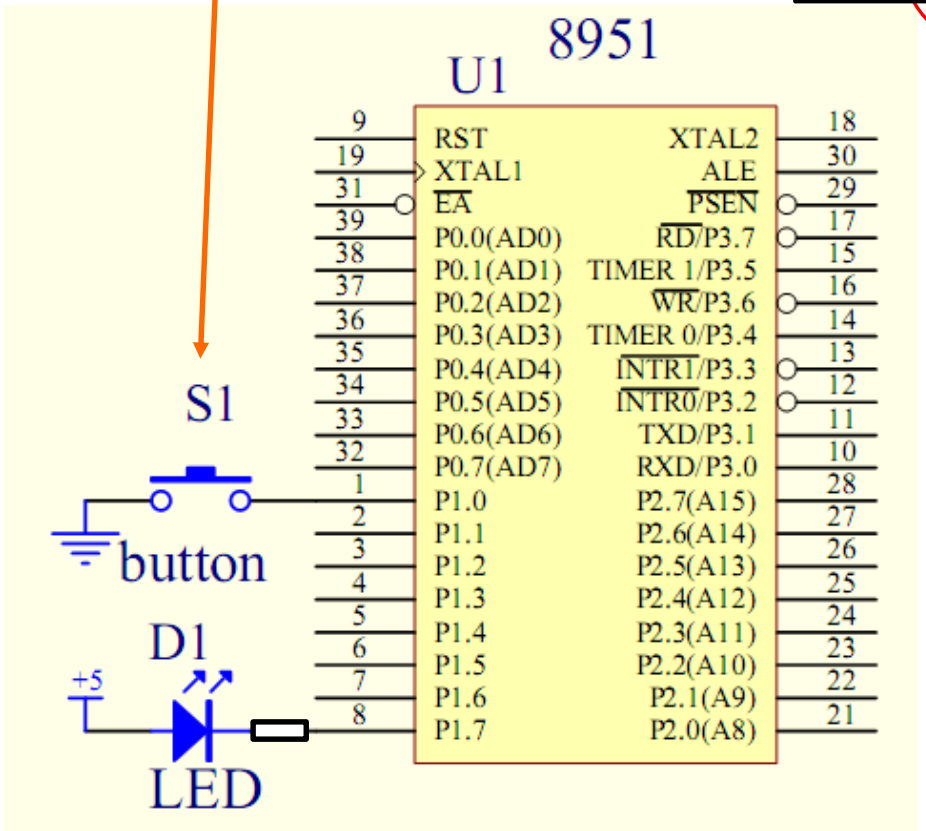
开关

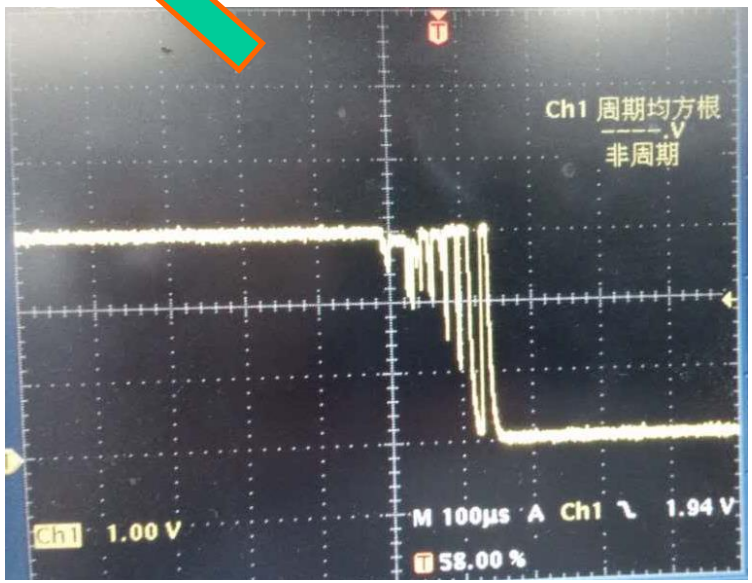
按键的抖动

轻触开关



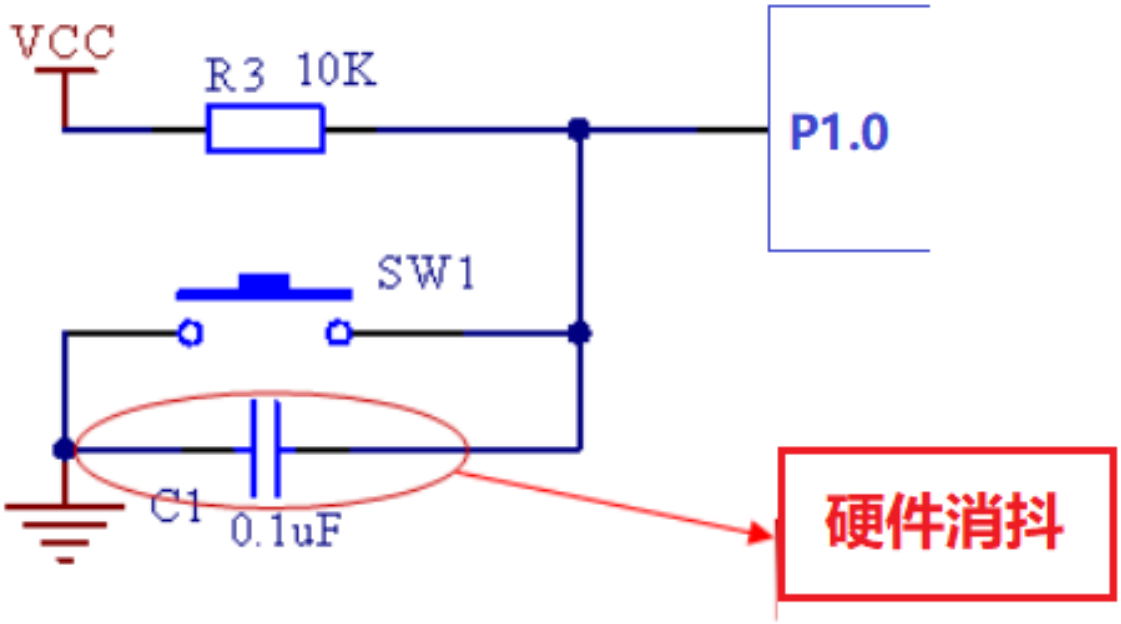

u
5V
抖动
t

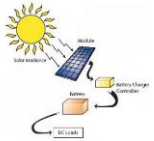

8951
U1
RST
XTAL1
EA
P0.0(AD0)
P0.1(AD1)
P0.2(AD2)
P0.3(AD3)
P0.4(AD4)
P0.5(AD5)
P0.6(AD6)
P0.7(AD7)
P1.0
P1.1
P1.2
P1.3
P1.4
P1.5
P1.6
P1.7
XTAL2
ALE
PSEN
RD/P3.7
TIMER 1/P3.5
WR/P3.6
TIMER 0/P3.4
INTR1/P3.3
INTR0/P3.2
TXD/P3.1
RXD/P3.0
P2.7(A15)
P2.6(A14)
P2.5(A13)
P2.4(A12)
P2.3(A11)
P2.2(A10)
P2.1(A9)
P2.0(A8)
S1
button
D1
+5
LED


Ch1 周期均方根
非周期
Ch1 1.00 V
M 100µs A Ch1 1.94 V
58.00 %

独立按键的消抖方法

1.硬件电路方法





独立按键的消抖方法

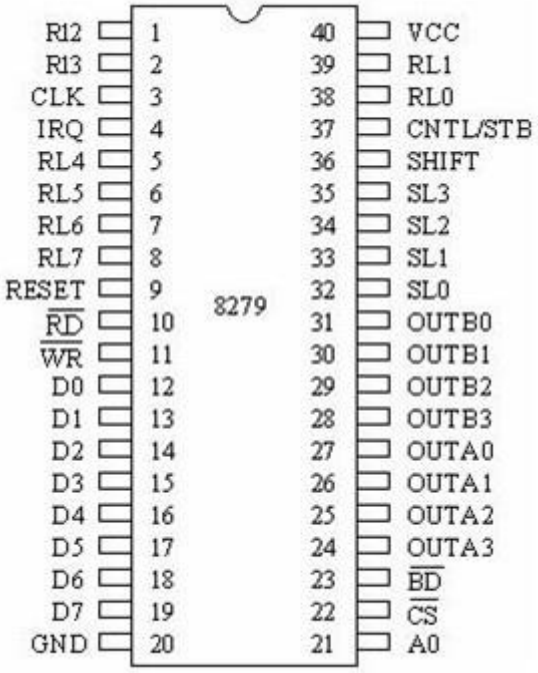
2.软件方法：多次检测

例：上图中，检测到按键则LED状态变化一次

```
LOOP:
        JB      P1.0,LOOP       ;;按下
        LCALL   DELAY10MS       ;;延时10ms，去抖
        JB      P1.0,LOOP       ;;按下
        CPL     P1.7            ;;LED灯转换
LOOP1:
        JNB     P1.0,LOOP1      ;;弹起
        LCALL   DELAY10MS       ;;去抖
        JNB     P1.0,LOOP1
        LJMP    LOOP
```

矩阵按键的检测

1、专用集成电路（8279）



Intel公司8279芯片是一种通用可编程键盘/显示器接口芯片.

对键盘部分提供扫描工作方式,能对64个按键键盘阵列不断扫描,自动消抖,自动识别出闭合的键并得到键号,能对双键或N键同时按下进行处理.

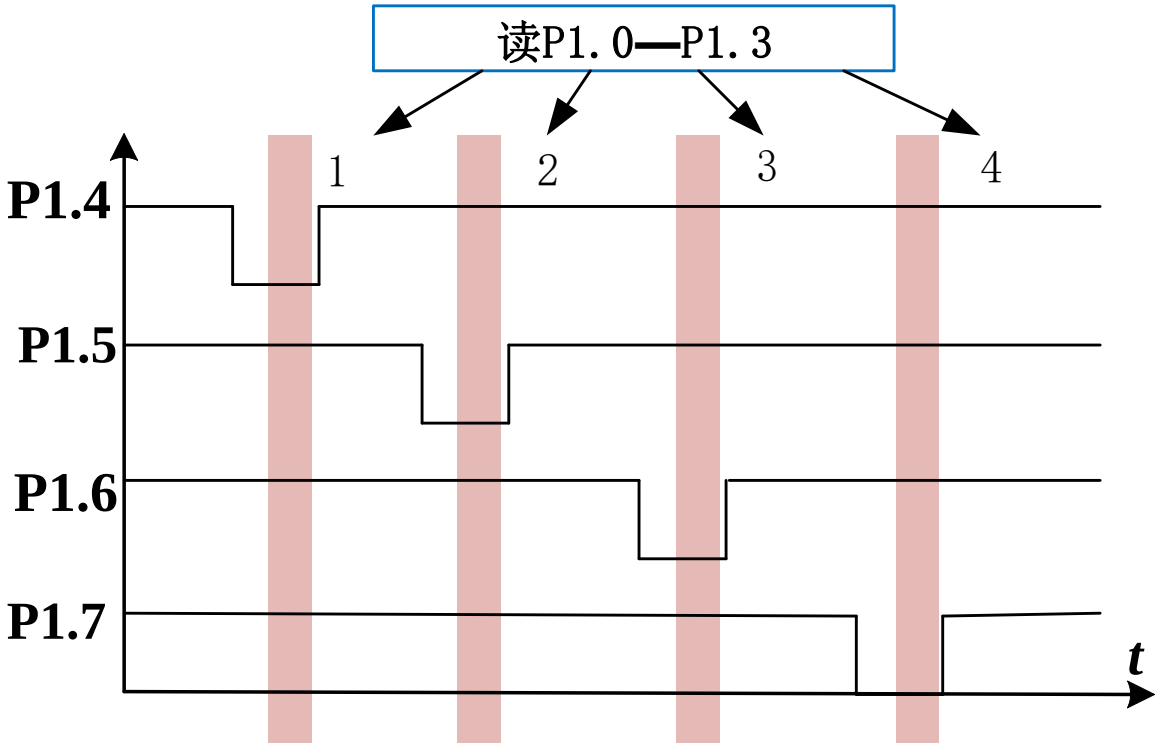
按扫描方式工作的显示接口,可显示多达16位的字符或数字.

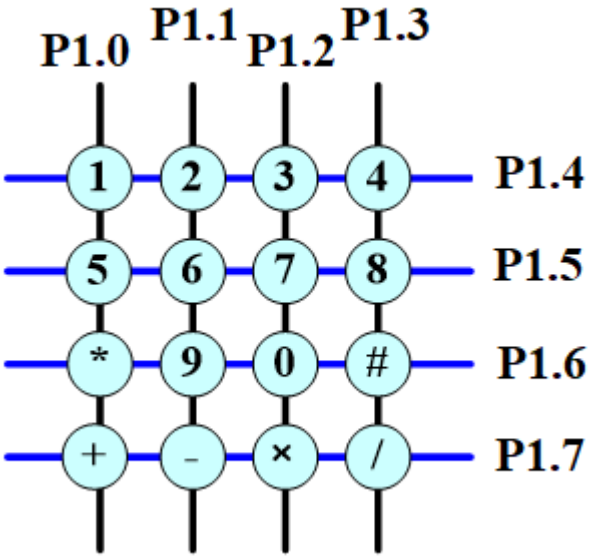
适合于键盘/数码管比较多的场合.

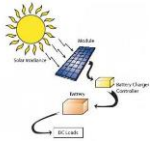
2、软件扫描

软件键盘扫描方法1：行扫描法

时序

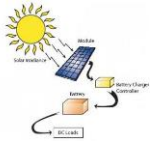






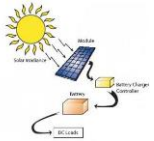
软件键盘扫描方法1：行扫描法

```
;;输出键码存放到 KEYCODE
KEY_SCAN1:
        MOV     P1,#0FFH
        MOV     KEYCODE,#0FFH  ;;输出键码
        CLR     P1.4                    ;; 检测第一行
        MOV     A,P1
        ANL     A,#0FH                  ;;取P1.0—P1.3
        CJNE    A,#0FH,CKLINE1
        SJMP    GLINE2                  ;;检测第二行
CKLINE1:
        MOV     B,A
        LCALL   DELAY10MS       ;;延时10ms，去抖
```



软件键盘扫描方法1：行扫描法

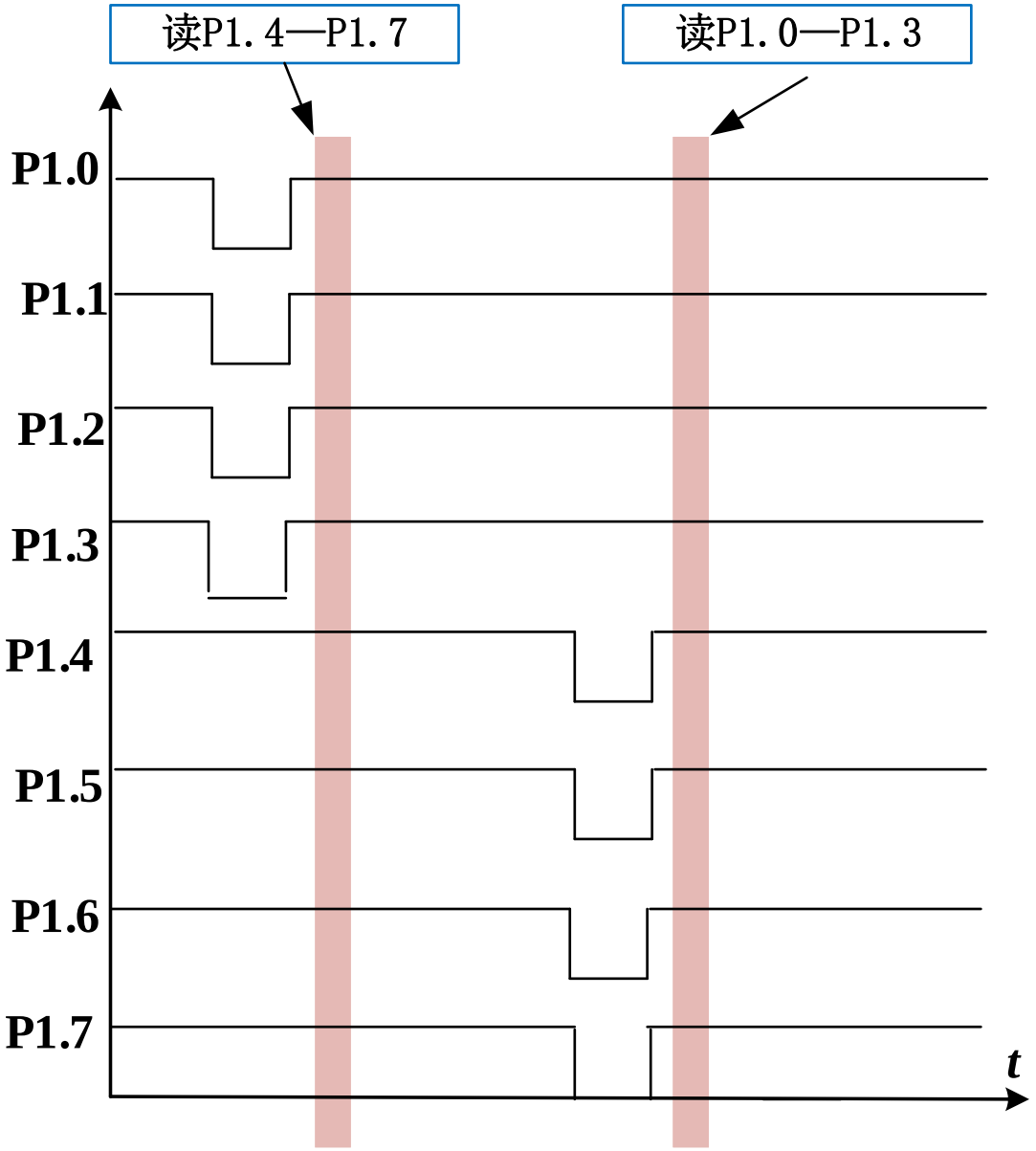
```
 MOV     A,P1
ANL      A,#0FH              ;;屏蔽高4位
CJNE     A,B,GLINE2          ;;相等则确认有按键
MOV      KEYCODE,#1
JNB      ACC.0,SCAN_OUT      ;;检测“1”键
INC      KEYCODE
JNB      ACC.1,SCAN_OUT      ;;检测“2”键
INC      KEYCODE
JNB      ACC.2,SCAN_OUT      ;;检测“3”键
INC      KEYCODE
SJMP     SCAN_OUT            ;;必定是“4”键
```

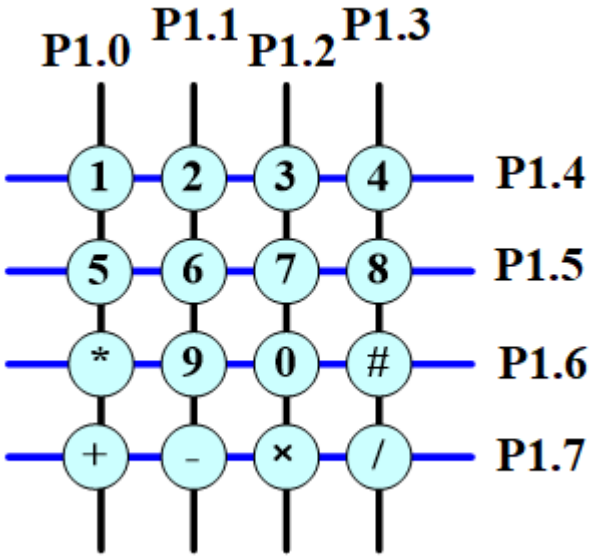


软件键盘扫描方法1：行扫描法

```
GLINE2：                        ;;检测第二行
        SETB    P1.4
        CLR     P1.4
        ……
GLINE3：                        ;;检测第三行
        SETB    P1.4
        CLR     P1.4
        ……
GLINE4：                        ;;检测第四行
        SETB    P1.4
        CLR     P1.4
        ……
SCAN_OUT:   RET
```

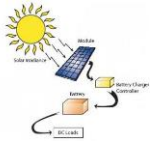
软件键盘扫描方法2：行列反转法





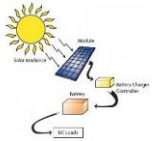
软件键盘扫描方法2：行列反转法

```
KEY_SCAN2:
        MOV     KEYCODE,#0FFH  ;;输出键码
        MOV     P1,#0F0H
        MOV     A,P1
        CJNE    A,#0F0H,NEXT1   ;;判断有无按键
        SJMP    NOKEY
NEXT1 :
        ANL     A,#0F0H     ;;取P1.4—P1.7（可省略）
        MOV     B,A
        MOV     P1,#0FH
        ANL     A,#0FH              ;;取P1.0—P1.3
        ORL     A,B
        LCALL   DELAY10MS           ;;延时10ms，去抖
```



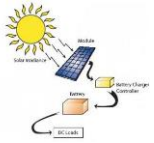
软件键盘扫描方法2：行列反转法

```
PUSH    ACC
MOV     P1,#0F0H
MOV     A,P1
ANL     A,#0F0H          ;;取P1.4—P1.7
MOV     B,A
MOV     P1,#0FH
ANL     A,#0FH           ;;取P1.0—P1.3
ORL     A,B
POP     B
CJNE    A,B,NOKEY
```



软件键盘扫描方法2：行列反转法

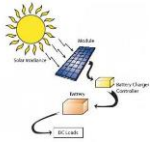
```
        MOV     DPTR,#TAB_KEY
        MOVC    A,@A+DPTR
        MOV     KEYCODE,A
NOKEY:
        RET
TAB_KEY:
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;0-10H
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;10-1FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;20-2FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;30-3FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;40-4FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;50-5FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;60-6FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0CH,0FFH,0FFH,0FFH,0DH,0FFH,0EH,0FH,0FFH ;70-7FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;80-8FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;90-9FH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;A0-AFH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0AH,0FFH,0FFH,0FFH,09H,0FFH,00H,0BH,0FFH  ;B0-BFH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;C0-CFH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,05H,0FFH,0FFH,0FFH,06H,0FFH,07H,08H,0FFH  ;D0-DFH
DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,01H,0FFH,0FFH,0FFH,02H,0FFH,03H,04H,0FFH  ;E0-EFH
```



地址

表格

00-07H	**0FFH**							
08-0FH								
10H-17H								
18H-1FH								
20H-27H								
28H-2FH								
30H-37H								
38H-3FH								
40H-47H								
48H-4FH								
50H-57H								
58H-5FH								
60H-67H								
68H-6FH								
70H-77H								
78H-7FH								



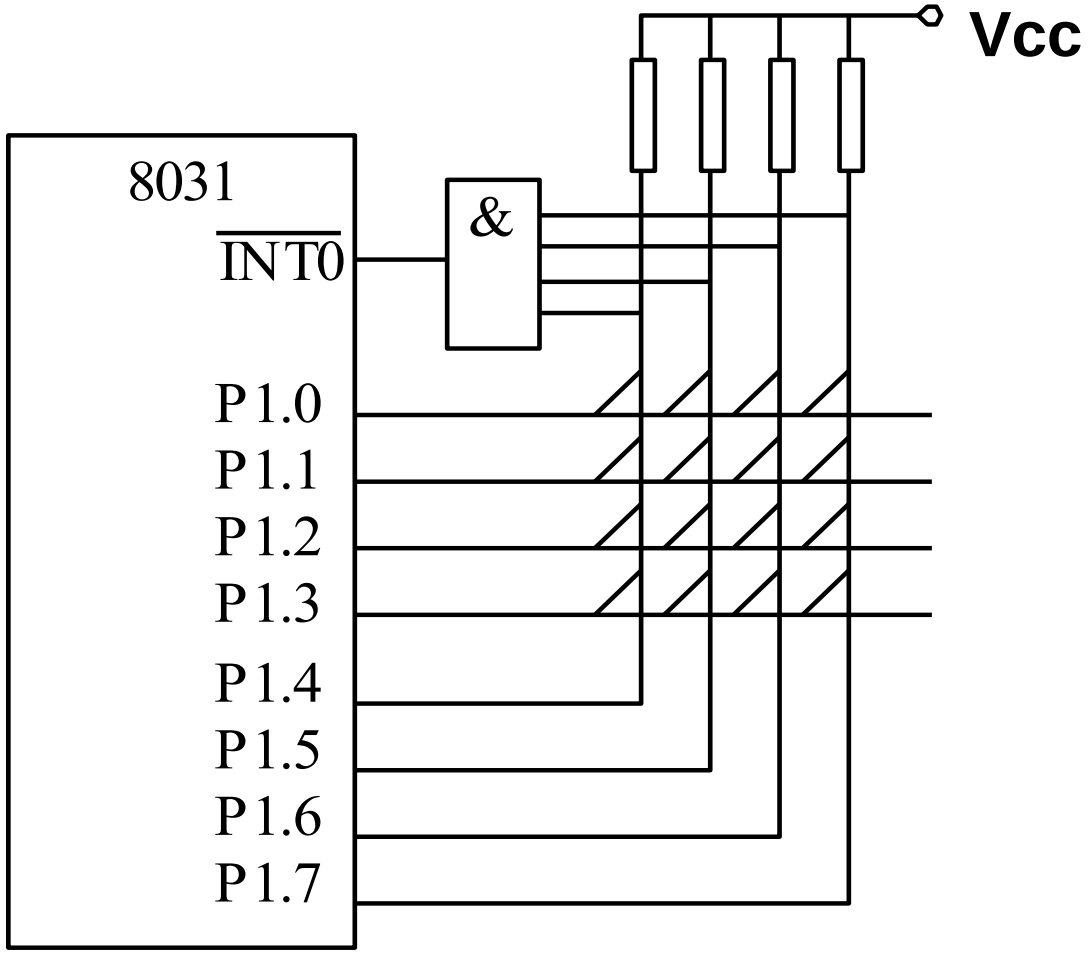
地址 表格

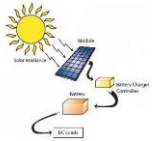
80H-87H	**0FFH**							
88H-8FH								
90H-97H								
98H-9FH								
0A0H-0A7H								
0A8H-0AFH								
0B0H-0B7H								
0B8H-0BFH								
0C0H-0C7H								
0C8H-0CFH								
0D0H-0D7H								
0D8H-0DFH								
0E0H-0E7H								
0E8H-0EFH						02	01	
0F0H-0F7H								
0F8H-0FFH								

键盘扫描程序设计

(1)中断方式

当无键按下时,CPU处理自己的工作,当有键按下时,产生中断请求,CPU转去执行键盘扫描子程序,并识别键号。

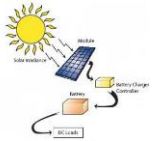




键盘扫描程序设计

(2)定时扫描方式

定时扫描方式就是每隔一段时间对键盘扫描一次,它利用单片机内部的定时器产生一定时间(例如10ms)的定时,当定时时间到就产生定时器溢出中断。CPU响应中断后对键盘进行扫描,并在有键按下时识别出该键,再执行该键的功能程序。



8.4-显示扩展

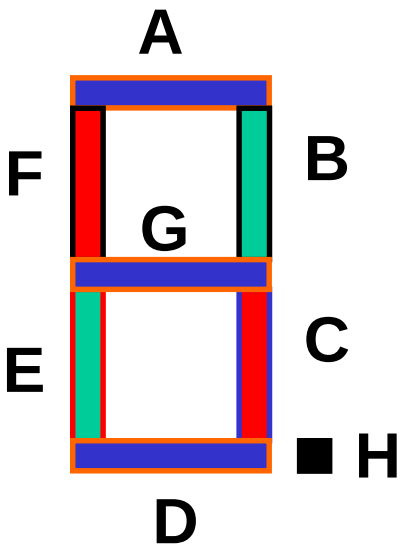
8.4.1 LED显示

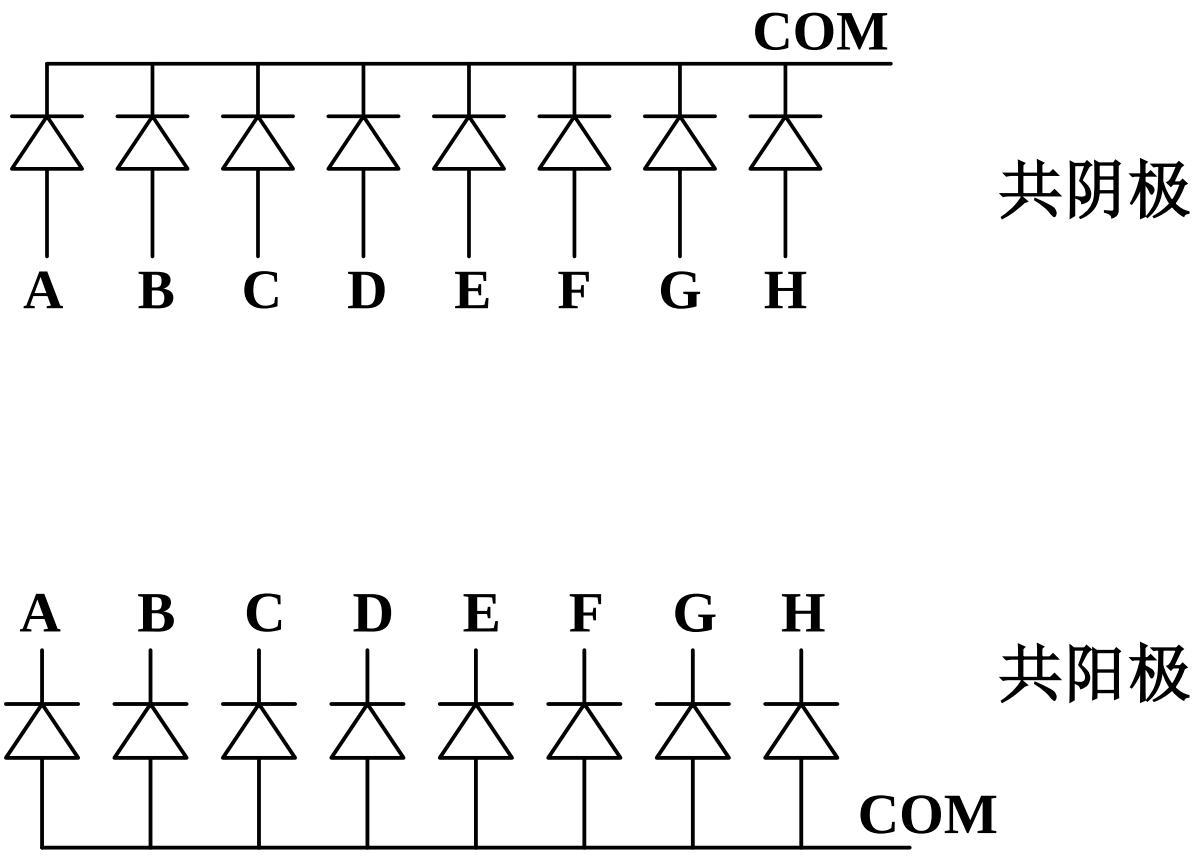
场合应用：指示灯，数码管，LED大屏，交通指示灯，LED照明

显示方式：

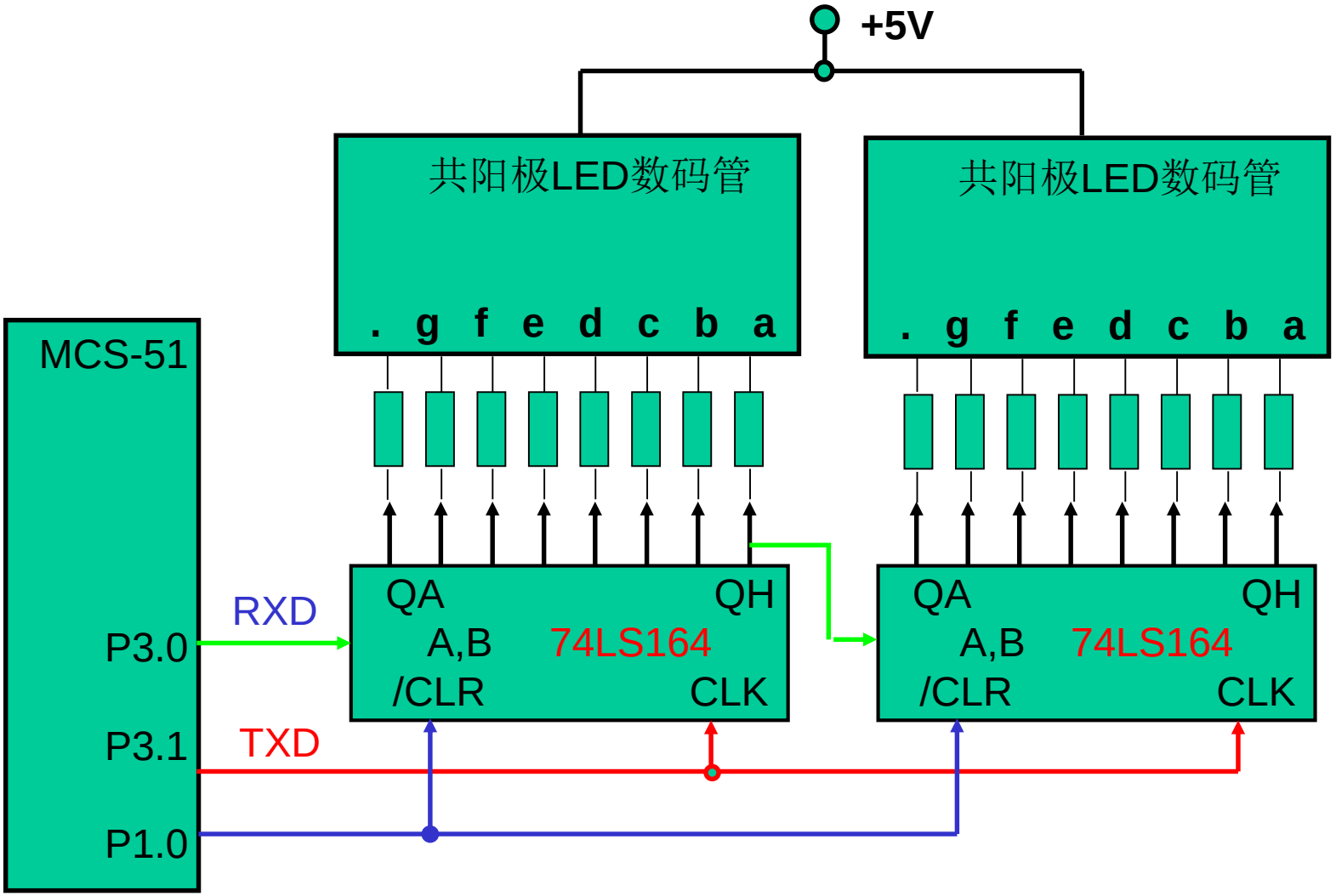
静态显示，动态显示

8段数码管结构

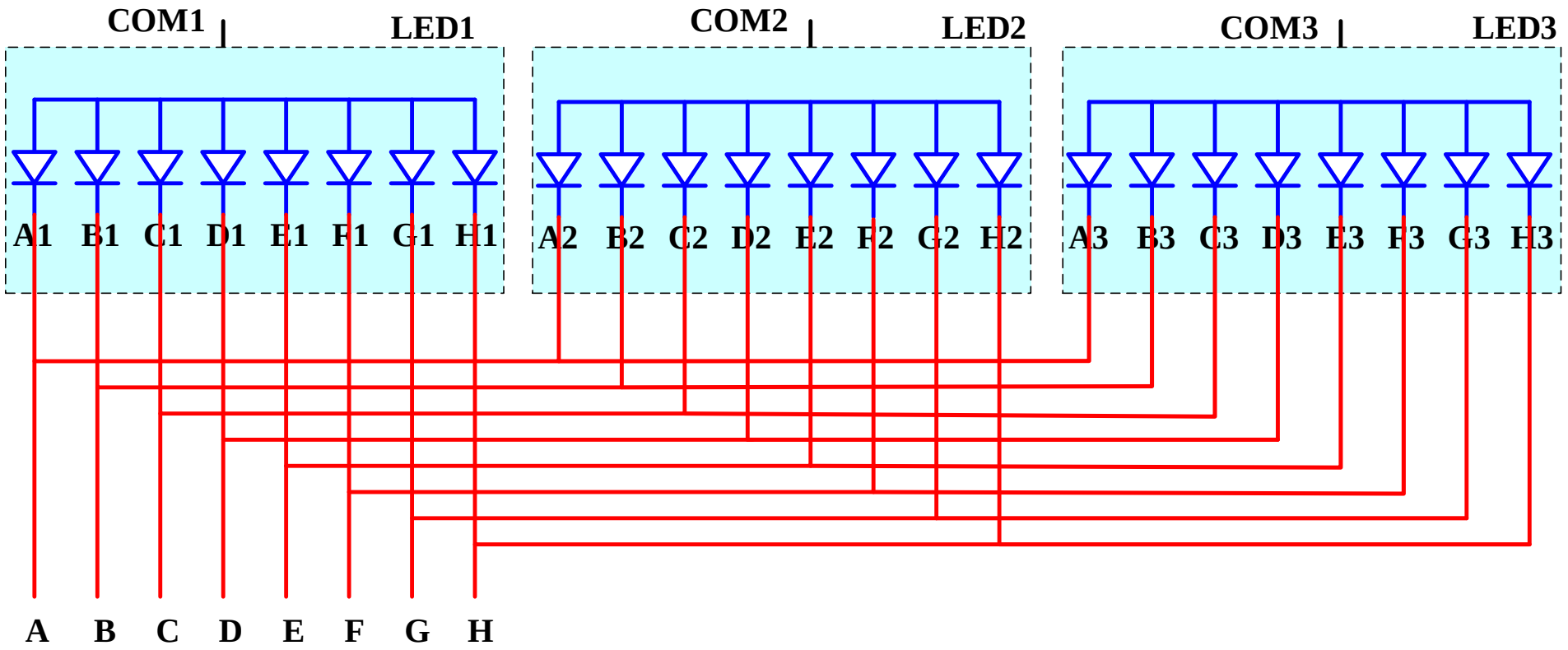




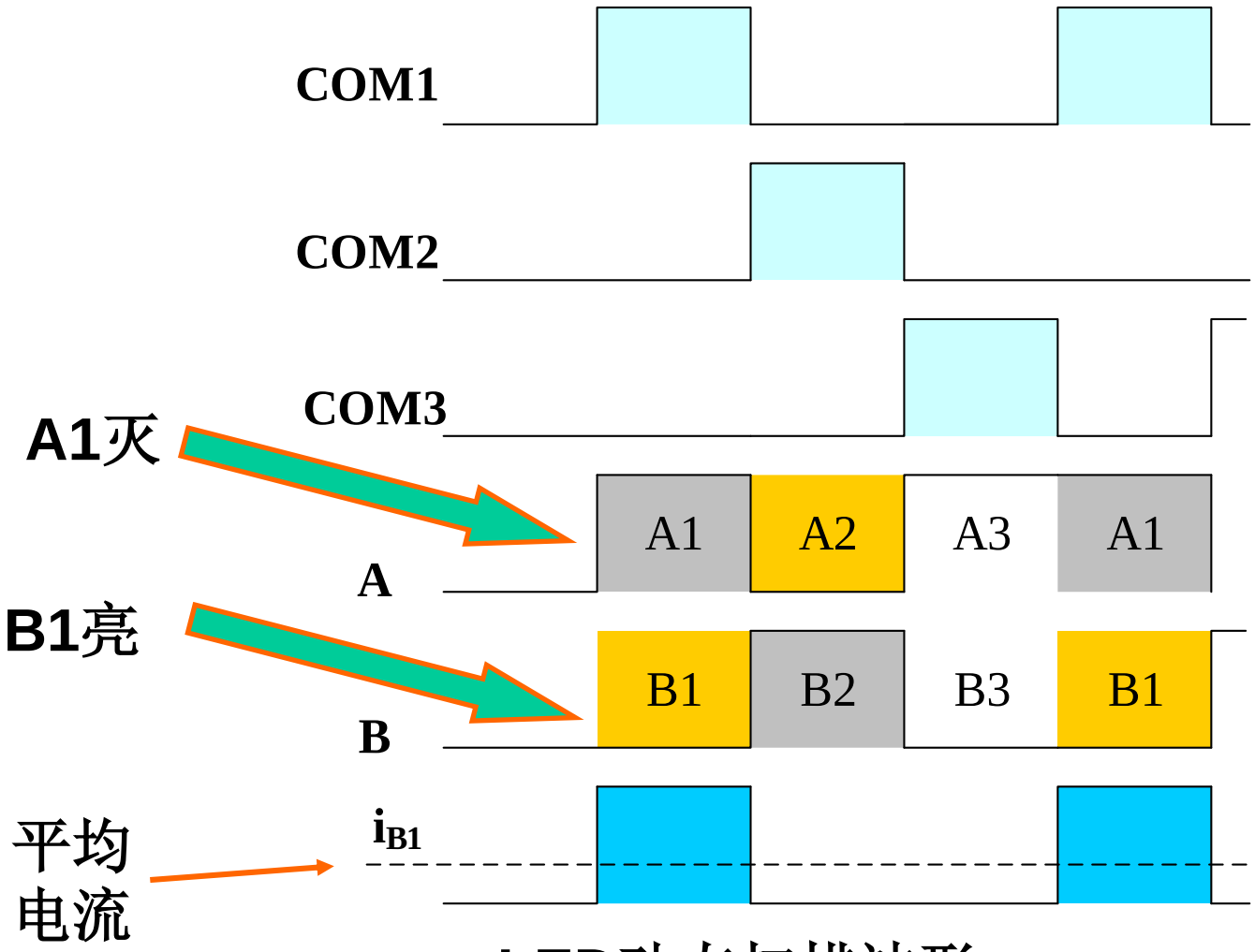
使用2片74LS164构成的16位输出扩展口



数码管动态显示


COM1
LED1
COM2
LED2
COM3
LED3
A1 B1 C1 D1 E1 F1 G1 H1
A2 B2 C2 D2 E2 F2 G2 H2
A3 B3 C3 D3 E3 F3 G3 H3
A B C D E F G H

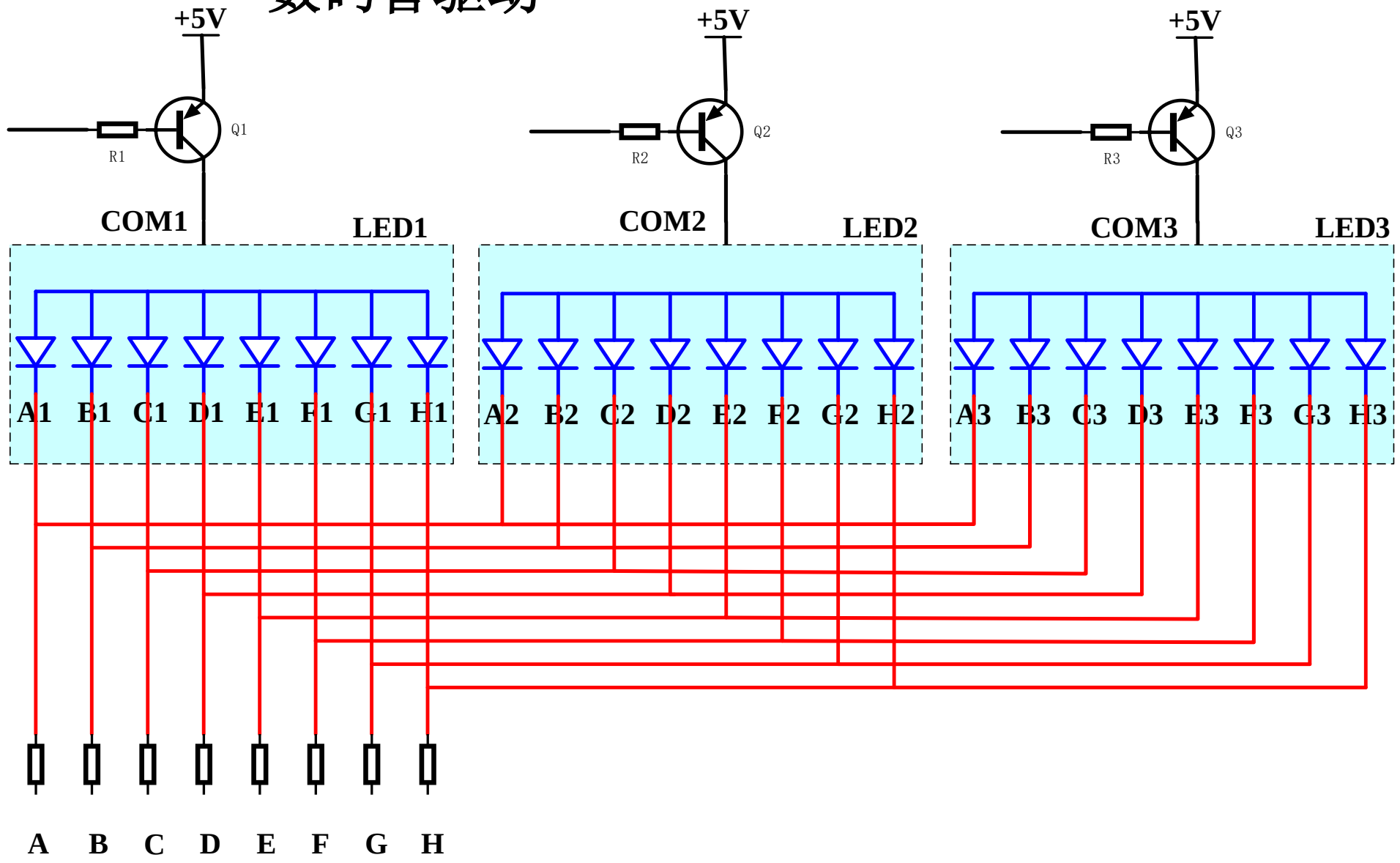
LED数码管动态显示



LED动态扫描波形

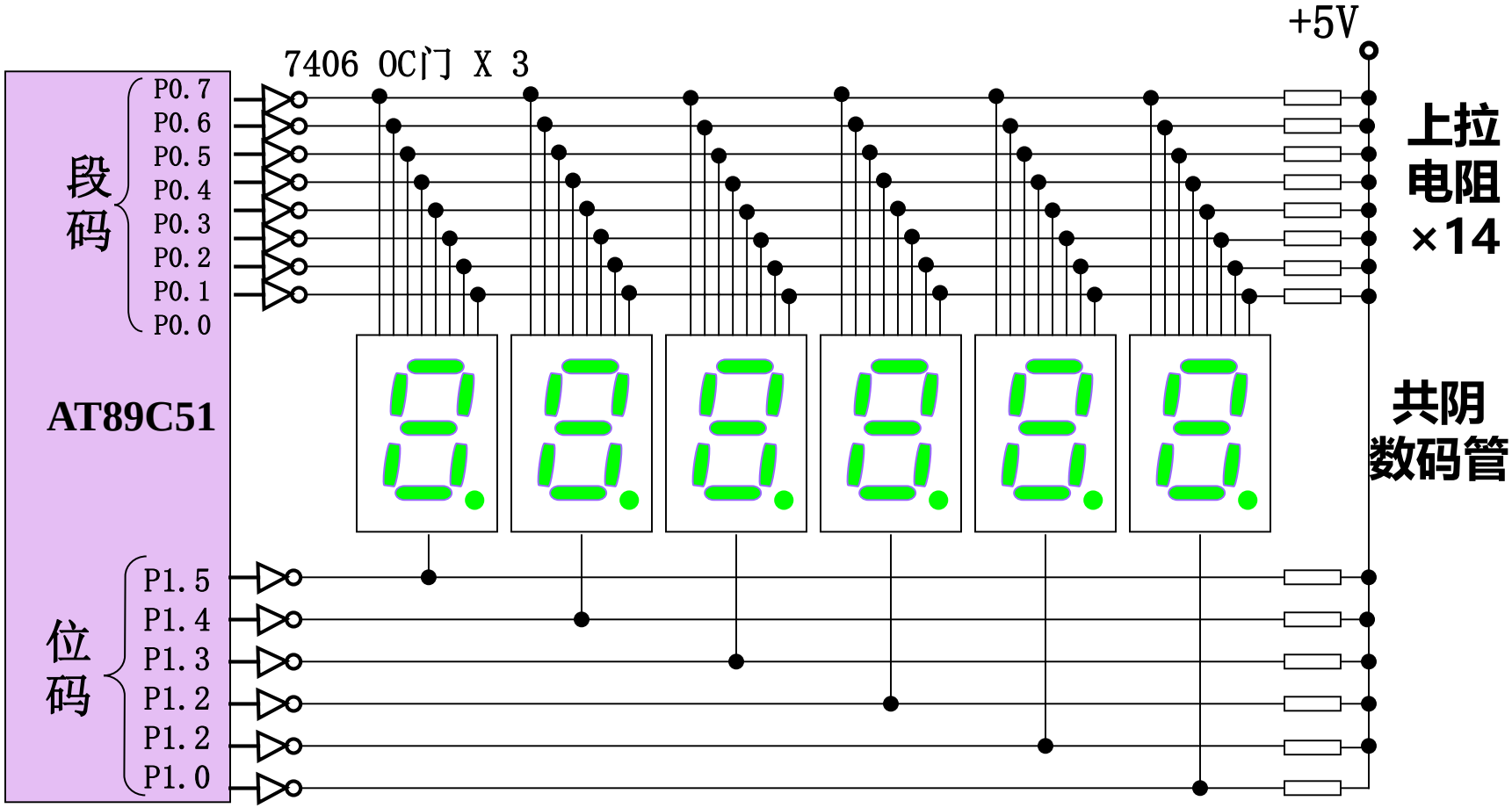
数码管动态显示

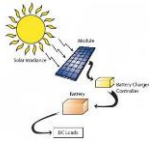
数码管驱动



例:动态显示举例

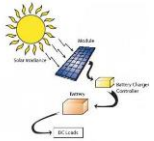
工作原理:从P0口送段代码,P1口送位选信号.段码虽同时到达6个LED,但一次仅一个LED被选中.利用“视觉暂留”,每送一个字符并选中相应位线,延时一会儿,再送/选下一个……循环扫描即可.





显示子程序设计：

```
DIR:
        MOV     DPTR,  #DSEG            ;数码管段码表首址
        MOV     R0,#60H                 ;待显缓冲区首地址
        MOV     R3,#01H                 ;个位的位选信号0000 0001
LD1:    MOV     A,@R0                   ;通过R0间接寻址
        MOVC    A,  @A+DPTR             ;查表
        MOV     P0,  A                  ;段码送到P0口
        MOV     P1,R3                   ;位码送到P1口
        LCALL DELAY                     ;调延时子程序
        INC     R0                      ;R0 指向下一字节7BH
        MOV     A,R3
        JB       ACC.5,LD2              ;判是否发完6个数？
        RL      A                       ;R1指向下一个位0000 0010
        MOV     R3,A                    ;位选信号存回R1
        SJMP    LD1                     ;跳去再显示下一个数
LD2:  RET                               ;发完6个数就返回
DSEG:DB  0C0H,0F9H,0A4H,0B0H,99H        ;共阳段码表,7406反相
                DB  92H,  82H,  0F8H,  80H,90H
```



例：时钟显示实验

实验内容:

1、在LED上显示实时时钟;

2、要求能够修改时钟：

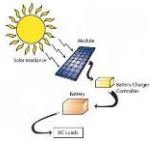
（1）正常情况下不显示光标；

（2）按确认键后显示光标；

（3）可通过左移和右移键修改位置的光标；

（4）按数字键修改光标对应位置的数字；

（5）禁止非法数字输入。

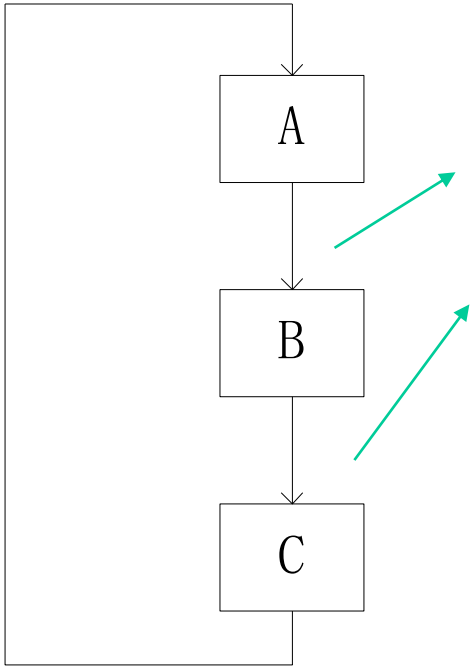


分时多任务模块化编程方法

- 原理：设定一最小工作周期（时间片），将所有任务进行分时处理。
- 方法：（1）划分任务模块
  （2）每一任务分时处理
  （3）任务中用状态机标识进度
- 特点：一般不采用DELAY子程序，而利用时间片延时。

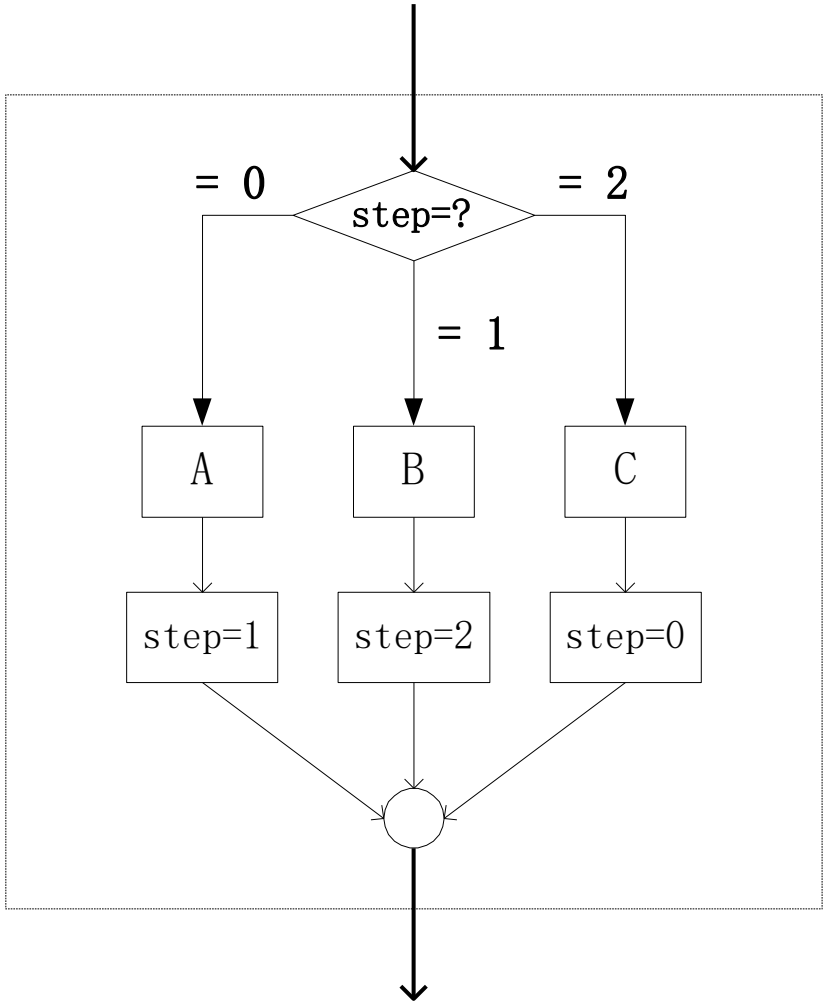
两种程序设计方法

包含A、B、C步骤的任务



允许插入
其他程序

顺序编程（单任务）



状态机法

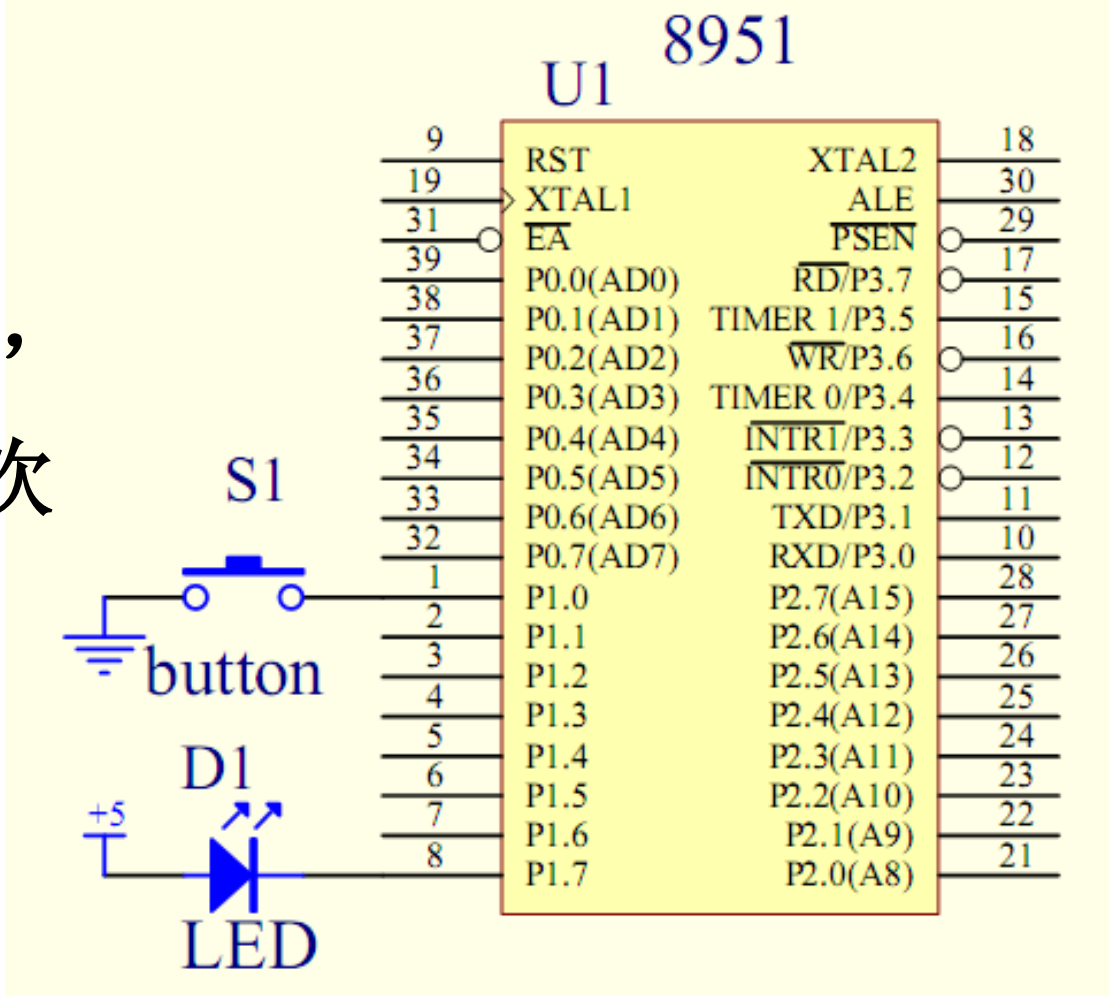
（适合多任务编程）

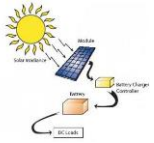
例：按键检测程序

功能：

每按一次按键**S1**，

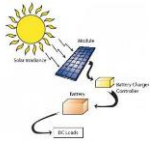
LED状态变化一次





顺序执行的程序

```
        LCALL   INI_CPU         ;;CPU初始化
LOOP:
        JB      P1.0,LOOP       ;;按下
        LCALL   DELAY10MS       ;;延时10ms，去抖
        JB      P1.0,LOOP       ;;按下
        CPL     P1.7            ;;LED灯转换
LOOP1:
        JNB     P1.0,LOOP1      ;;弹起
        LCALL   DELAY10MS       ;;去抖
        JNB     P1.0,LOOP1
        LJMP    LOOP
```



◆顺序编程的缺点：

独占CPU处理时间

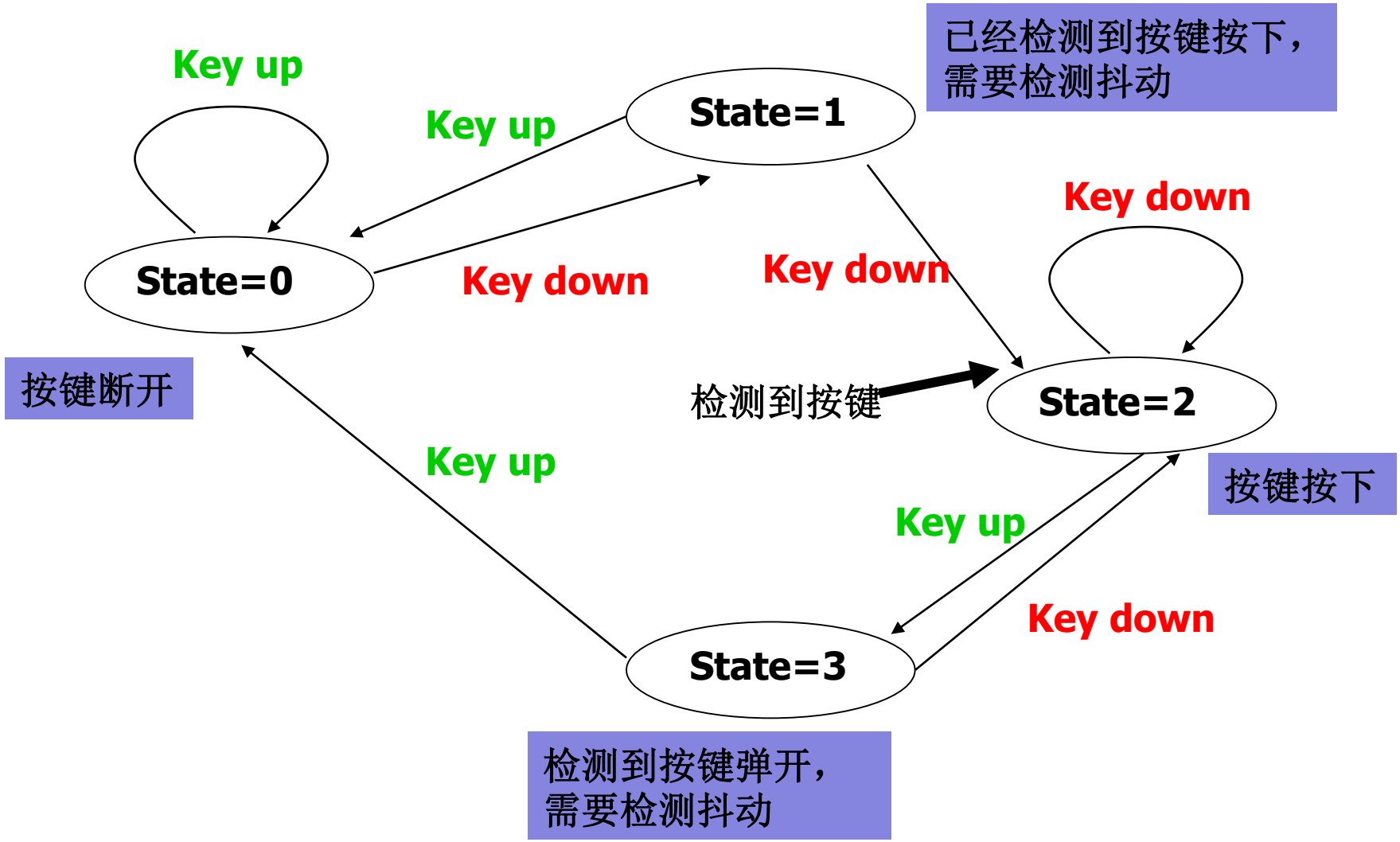
不容易加入其他程序

采用分时处理编程方法：

一般用有限状态机设置当前状态标记，

在下一次循环接着上次任务继续执行。

有限状态机（KEY_STATE）



分时处理程序

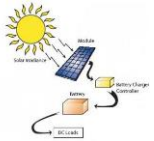
```
        KEY_STATE       EQU         20H     ;;按键状态
        B_KEY           BIT         00H     ;;按键标志
;;主程序
        LCALL       INI_TIMER1_10MS
        MOV   KEY_STATE,#0
        CLR    B_KEY
               ;;定时器10ms定时
LOOP:           ;;循环体
    {   LCALL       KEY_SCAN            ;;键盘扫描子程序
    {   LCALL       DIS_LED            ;;显示子程序
;;10ms循环
        JNB         TF1,$
        CLR         TF1
       LCALL        INI_TIMER1_10MS
        LJMP        LOOP
```

键盘扫描程序(1)

```
KEY_SCAN：
;;输入：P1.0
;;输出：B_KEY
        MOV     A,KEY_STATE             ;;读当前State状态
        CJNE    A,#0,STATE1
STATE0:
        JB      P1.0,KEY_EXIT
        MOV     KEY_STATE,#1            ;;第一次发现键按下，转状态1
KEY_EXIT:
        RET
STATE1:
        CJNE    A,#1,STATE2
        JB      P1.0,KEY_S2
        MOV     KEY_STATE,#2
        SETB    B_KEY                   ;;输出按键标志
        RET
KEY_S2:
        MOV     KEY_STATE,#0
        RET
```

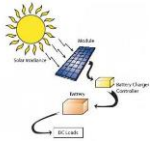
键盘扫描程序(2)

```
STATE2:
        CJNE    A,#2,STATE3
        JB      P1.0,STATE_S2
;;按下，state保持不变
        RET
STATE_S2:
;;      松开
        MOV     STATE,#3
        RET
STATE3:
        CJNE    A,#3,STATE2
        JB      P1.0,STATE_S3
        MOV     STATE,#2
        RET
STATE_S3:
        MOV     STATE,#0
        RET
```



显示程序

```
;;输入变量：B_KEY
;;输出：LED显示
DIS_LED:
        JNB         B_KEY,DIS_RET
        CPL         P1.7
        CLR         B_KEY
        RET
DIS_RET:
        RET
```

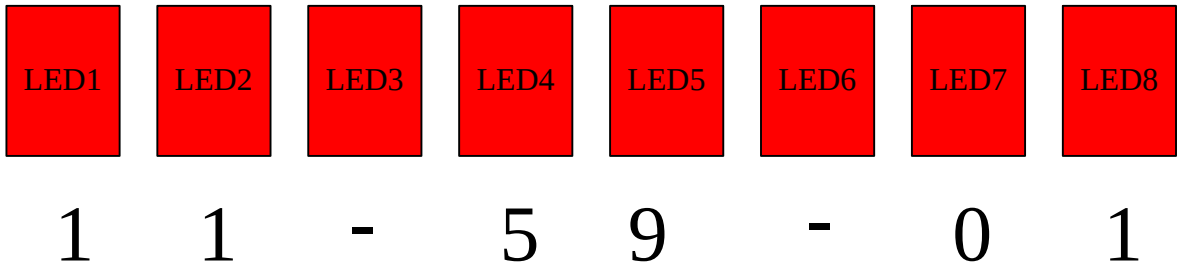


模块化子程序注释要求：

- 实现功能
- 输入变量名及其含义
- 输出变量名及其含义
- 使用或影响的变量
- 每时间片最大占用时间
- 使用堆栈数目

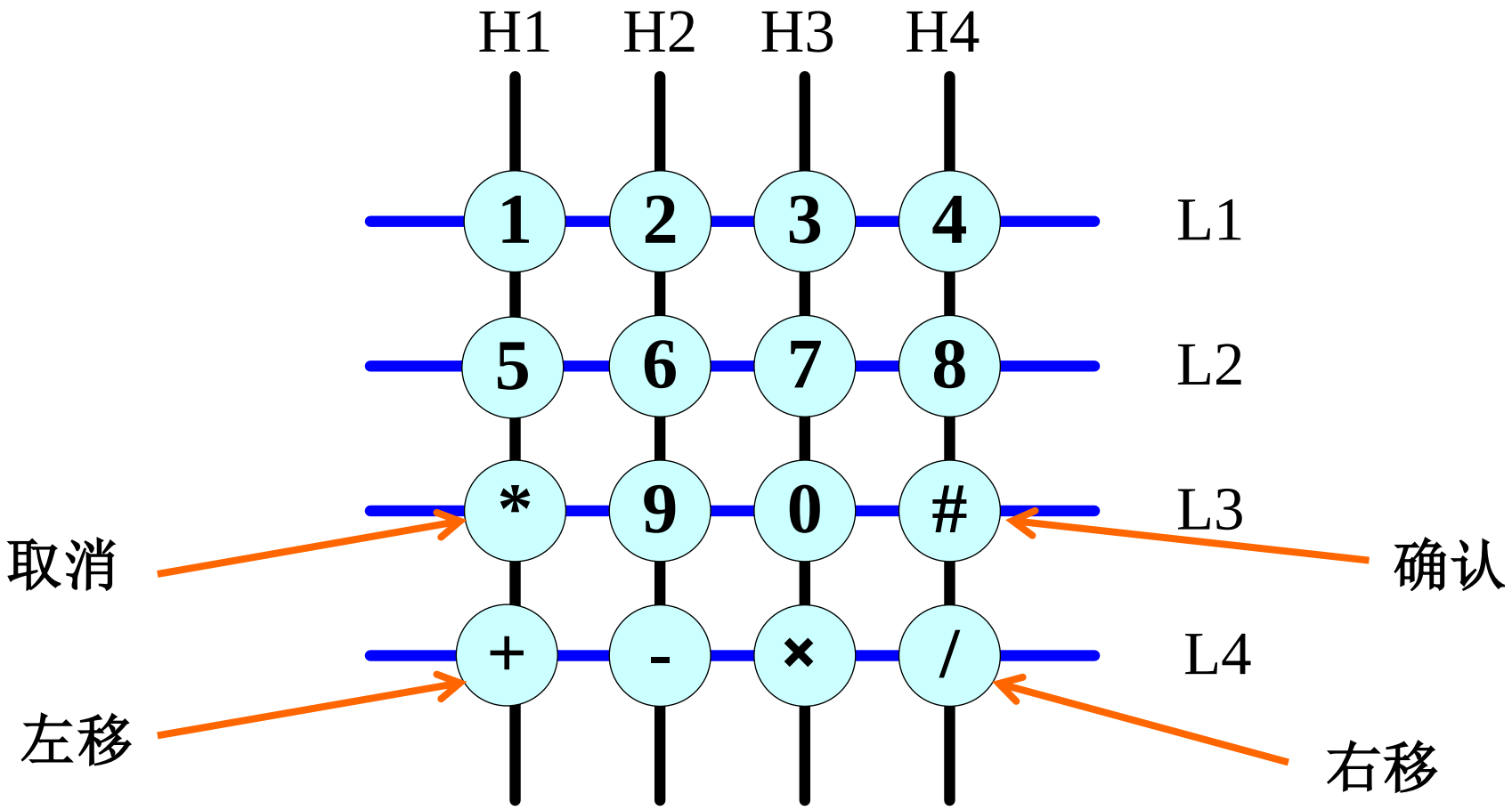
实验内容:

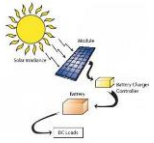
时钟显示方式:



光标设计：亮0.5秒，灭0.5秒(或显示“-”)

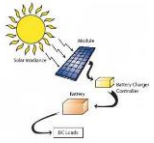
键盘定义





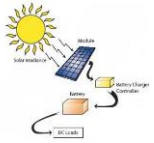
主程序

```
 ORG     0100H
BEGIN:
        MOV     SP,#70H
;;光标初始化位置
        MOV     CURSOR,#0
        CLR     B_CURSOR
;;按键初始化
        MOV     KEYCODE,#0FFH
        MOV     LASTCODE,#0FFH
        MOV     N_SAME,#0
;;LED显示初始化
        MOV     R1,#8
        MOV     R0,#LED_BUF
CLRLOOP:
        MOV     @R0,#0FH
        INC     R0
        DJNZ    R1,CLRLOOP
;;显示时间值初始化
        LCALL   INI_TIMER
LOOP:
        SJMP    LOOP
```



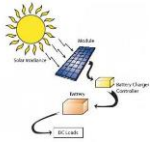
;;定时器T0初始化

```
INI_TIMER:
        MOV     T_2MS,#0
        MOV     T_100MS,#0
        MOV     SECOND,#0
        MOV     MINUTE,#59
        MOV     HOUR,#11
        MOV     TMOD,#01H
        ;;TIMER0 MODE1
        MOV     TCON,#0
        MOV     TH0,#0F8H
        MOV     TL0,#30H
        ;;定时2mS
        SETB    TR0
        SETB    ET0
        SETB    EA
        RET
```



;; 定时器中断

```
TIMER0_INT:
        PUSH    ACC
        PUSH    PSW
        LCALL TIMER
;;时钟运行
        LCALL   DISP_TIMER
;;时钟放入显示缓冲
        LCALL   KEY_SCAN
;;键盘扫描
        LCALL   DO_KEY
;;根据键码值运行相应操作
        LCALL   SET_CURSOR
;;显示光标
        LCALL   DISP_LED
;;LED显示
        POP     PSW
        POP     ACC
        RETI
```

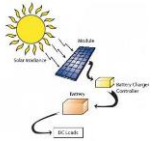


;;内部时钟运行

```
;;INPUT:NONE
;;OUTPUT: T_2MS, ,T_100MS
    SECOND,MINUTE,HOUR
;;INFLUENCED:ACC

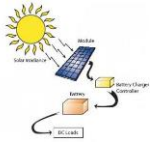
TIMER:
      MOV    TH0,#0F8H
      MOV    TL0,#30H            ;;2ms
;;2ms计数加1
      INC    T_2MS
      MOV    A,T_2MS
      CLR    C
      SUBB   A,#50
      JC     TIMER0_RET
      MOV    T_2MS,A
```

```
;;100ms计数加1
      INC    T_100MS
      MOV    A,T_100MS
      CLR    C
      SUBB   A,#10
      JC     TIMER0_RET
      MOV    T_100MS,A
;;秒计数加1
      INC    SECOND
      MOV    A,SECOND
      CLR    C
      SUBB   A,#60
      JC     TIMER0_RET
      MOV    SECOND,A
```



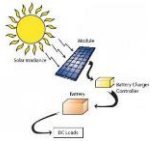
;;内部时钟运行

```
;;分钟计数加1
        INC     MINUTE
        MOV     A,MINUTE
        CLR     C
        SUBB    A,#60
        JC      TIMER0_RET
        MOV     MINUTE,A
;;小时计数加1
        INC     HOUR
        MOV     A,HOUR
        SUBB    A,#12
        JC      TIMER0_RET
        MOV     HOUR,A
TIMER0_RET:
        RET
```



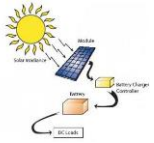
;;键盘扫描

```
;;INPUT:P1，P2
;;OUTPUT:KEYCODE
;;INFLUENCED:LASTCODE-上次采样的键码
;; N_SAME -采样到相同键码的次数
;;DPTR
;;状态：LASTCODE， N_SAME
KEY_SCAN:
      MOV    P1,#0F0H
      NOP
      MOV    A,P1
      ANL    A,#0F0H
      MOV    B,A
      MOV    P1,#0FH
      NOP
      MOV    A,P1
      ANL    A,#0FH
      ORL    A,B
      MOV    DPTR,#TAB_KEY
      MOVC   A,@A+DPTR
      CJNE   A,LASTCODE,GET_NEW_KEY
      MOV    A,N_SAME
      CJNE   A,#3,GET_NEXT
;;新键码
      MOV    KEYCODE,LASTCODE
KEY_EXIT:
      INC    N_SAME
      RET
GET_NEXT:
      JC     KEY_EXIT
      RET
GET_NEW_KEY:
      MOV    LASTCODE,A
      MOV    N_SAME,#0
      RET
```



;; 键盘扫描

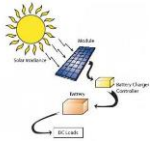
```
TAB_KEY:
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;0-10H
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;10-1FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;20-2FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;30-3FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;40-4FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;50-5FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH ;60-6FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0CH,0FFH,0FFH,0FFH,0DH,0FFH,0EH,0FH,0FFH ;70-7FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;80-8FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;90-9FH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;A0-AFH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0AH,0FFH,0FFH,0FFH,09H,0FFH,00H,0BH,0FFH  ;B0-BFH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;C0-CFH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,05H,0FFH,0FFH,0FFH,06H,0FFH,07H,08H,0FFH  ;D0-DFH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,01H,0FFH,0FFH,0FFH,02H,0FFH,03H,04H,0FFH  ;E0-EFH
    DB    0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH,0FFH  ;F0-FFH
```



;; LED动态扫描

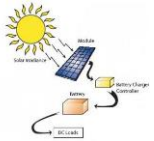
```
;;INPUT:LED_BUF
;;OUTPUT:
;;INFLUENCED:Acc，DPTR
;; 状态：N_DISP（0-7）
DISP_LED:
      MOV    A,#LED_BUF
      ADD    A,N_DISP ;;第几次扫描
      MOV    R0,A
      MOV    A,@R0
      MOV    DPTR,#TAB_LED
      ANL    A,#0FH
      MOVC   A,@A+DPTR
      MOV    P0,A
      MOV    A,#0FFH
      CLR    C
      INC    N_DISP
      MOV    R0,N_DISP
LOOP_D1:
      RLC    A
      DJNZ   R0,LOOP_D1
      MOV    P2,A
      ANL    N_DISP,#07H
      RET


TAB_LED:
DB     28H,7EH,0A2H,62H,74H
DB     61H,21H,7AH,20H,60H
DB     0FFH,0FFH,0FFH,0FFH,0F7H,0EFH
```



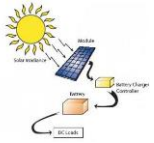
;显示时间，将显示数据放入显示缓冲区

```
;;INPUT:SECOND,MINUTE,HOUR
;;OUTPUT:LED_BUF
;;INFLUENCED:ACC，B，R0
DISP_TIMER:
    MOV    R0,#LED_BUF
    MOV    A,HOUR
    MOV    B,#10
    DIV    AB
    MOV    @R0,A
    INC    R0
    MOV    @R0,B
    INC    R0          ;;DISPLAY
  HOUR
    MOV    @R0,#0EH      ;;-
    INC    R0
    MOV    A,MINUTE
    MOV    B,#10
    DIV    AB
MOV    @R0,A
    INC    R0
    MOV    @R0,B
    INC    R0
;;DISPLAY MINUTE
    MOV    @R0,#0EH      ;;-
    INC    R0
    MOV    A,SECOND
    MOV    B,#10
    DIV    AB
    MOV    @R0,A
    INC    R0
    MOV    @R0,B
    ;;DISPLAY SECOND
    RET
```



;按键处理

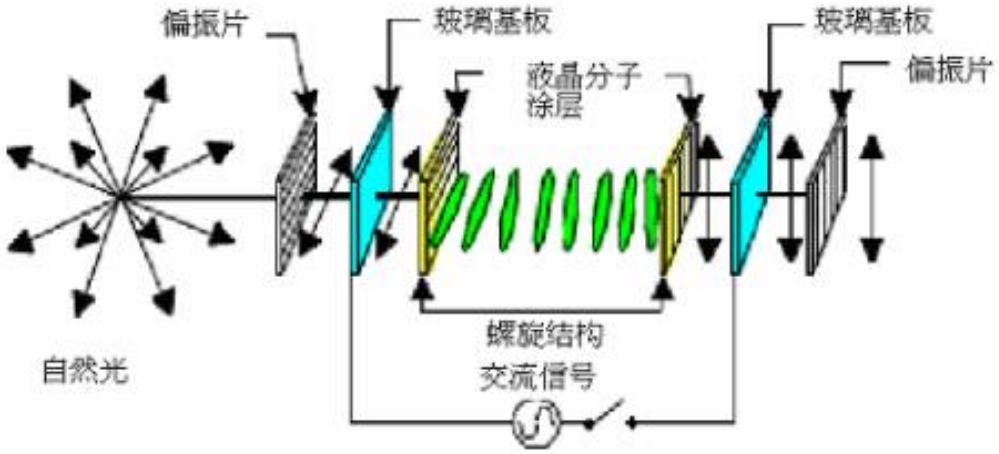
```
;;INPUT:KEYCODE
;;OUTPUT:
;;INFLUENCED:ACC
;;
DO_KEY:
      MOV    A,KEYCODE
      CJNE   A,#0FFH,KEY_0
      RET
KEY_0:
      CJNE   A,#0FH,KEY_1
      CPL    B_CURSOR
      SJMP   KEY_RET
KEY_1:
    NOP
KEY_RET:
      MOV    KEYCODE,#0FFH
      RET
```



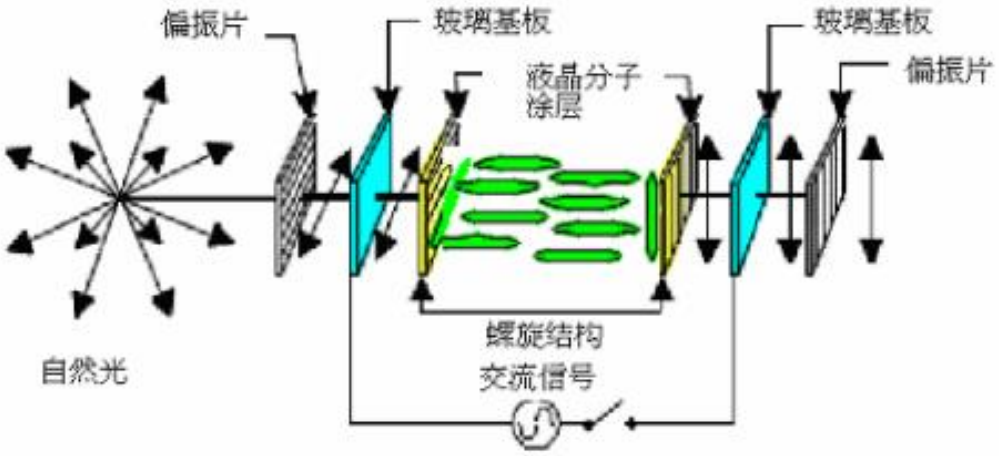
;设置光标

```
;;INPUT:B_CURSOR,CURSOR
;;OUTPUT:LED_BUF
;;INFLUENCED:ACC
SET_CURSOR:
        JNB     B_CURSOR,CURSOR_OUT
        MOV     A,T_100MS
        CLR     C
        SUBB    A,#5
        JC      CURSOR_OUT
        MOV     A,CURSOR
        ANL     A,#7
        ADD     A,#LED_BUF
        MOV     R0,A
        MOV     @R0,#0FH
CURSOR_OUT:
        RET
```

8.4.2 LCD(Liquid Crystal Display)显示



玻璃基板间电压= 0V



玻璃基板间电压= 交流电压

液晶分子是介于固体和液体之间的一种棒状结构的大分子物质。

主要类型:

扭转式向列型（Twisted Nematic；TN）

超扭转式向列型（Super Twisted Nematic；STN）

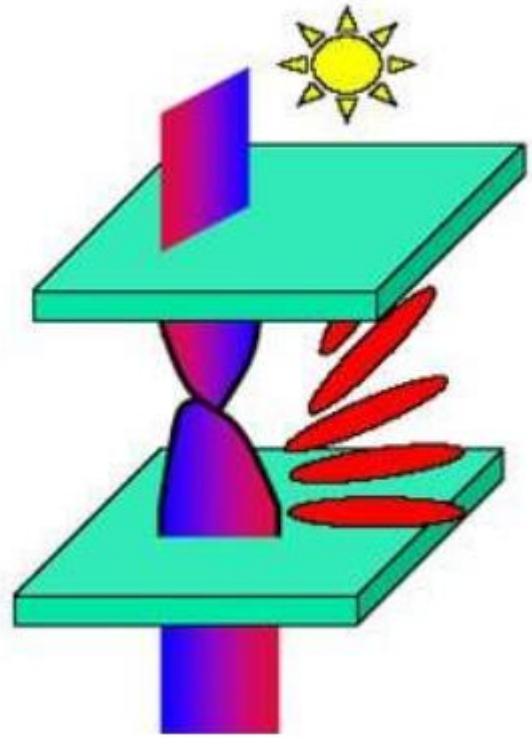
薄膜晶体管型（Thin Film Transistor；TFT）

特点:

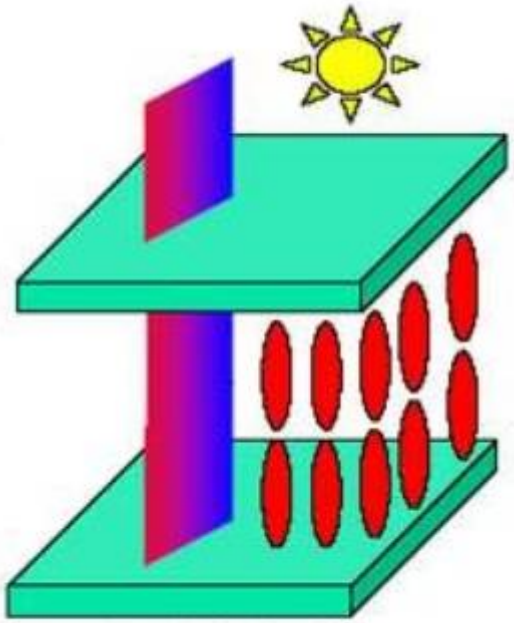
低功耗

被动发光

TN型LCD显示原理



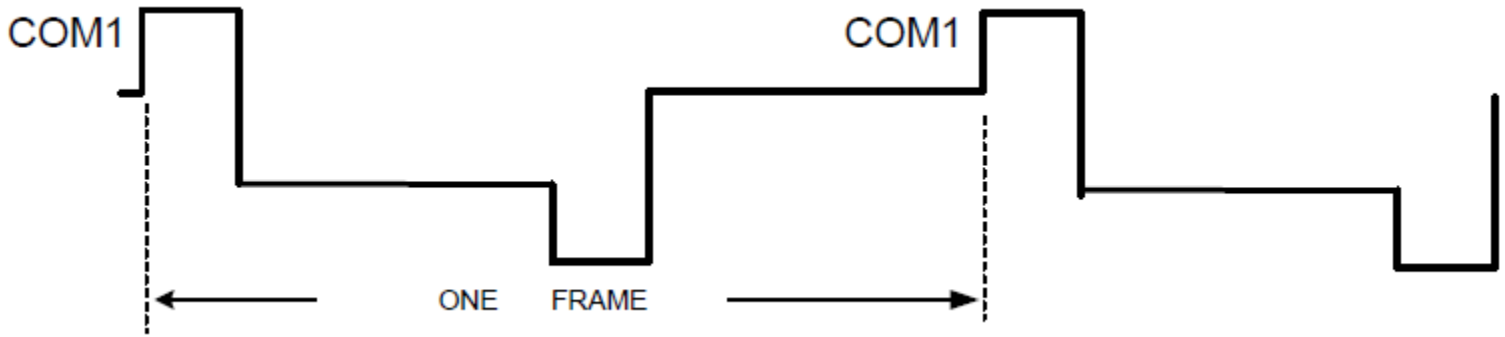
不加电压时透射
白色



加电压时不能透射
黑色

液晶驱动电压要求

- **交流电压：** 液晶分子是需要交流信号来驱动的，长时间的直流电压加在液晶分子两端，会影响液晶分子的电气化学特性，引起显示模糊，寿命的减少，其破坏性为不可恢复；



液晶驱动电压要求

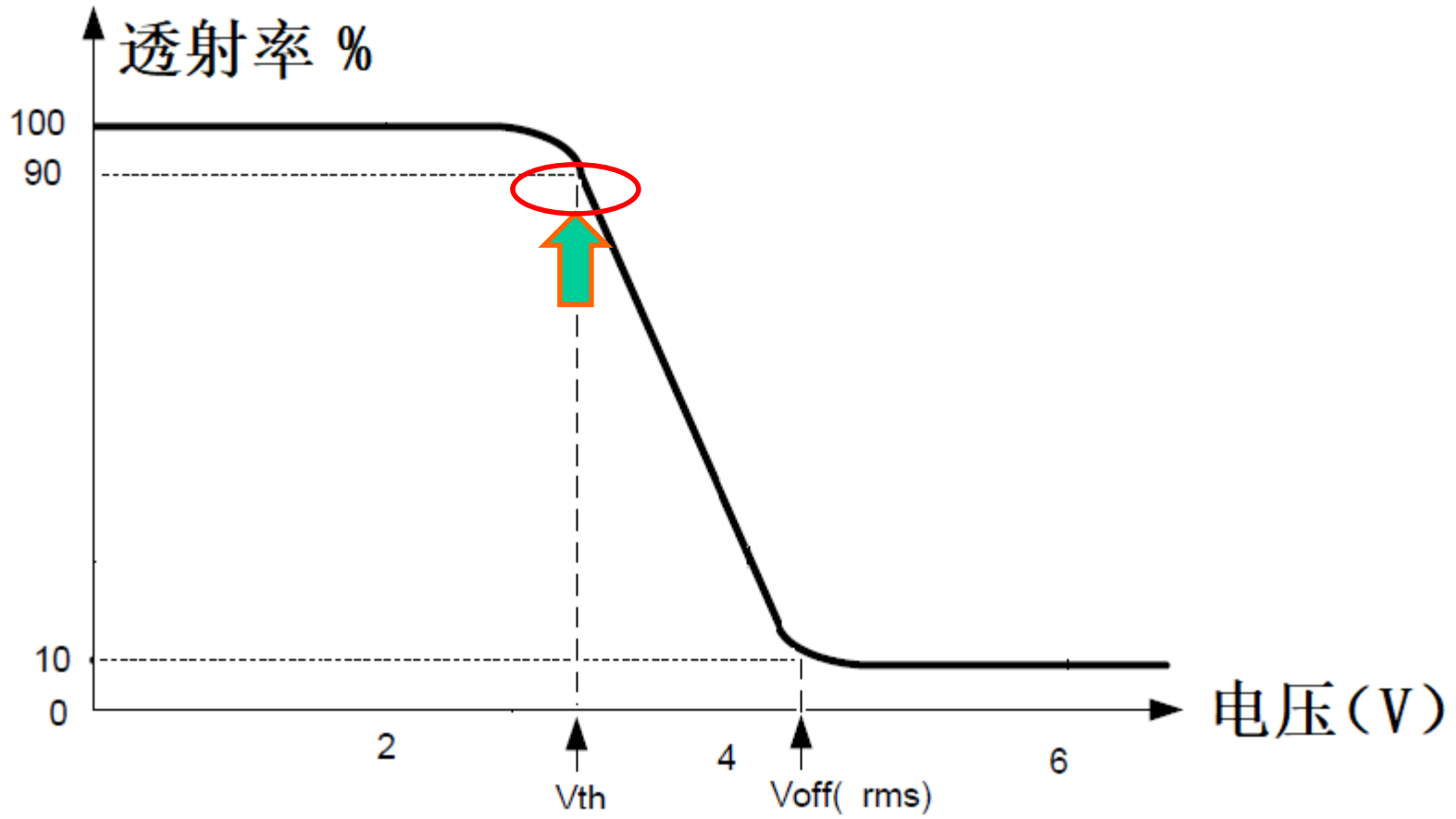
- **扫描频率：** 液晶驱动的交流电压频率一般在 60～100Hz 之间，具体是依据LCD Panel 的面积和设计而定，频率过高，会导致驱动功耗的增加，频率过低，会导致显示闪烁。

- **亮/暗显示：** 液晶分子是一种电压积分型材料，它的扭曲程度(透光性)仅仅和极板间电压的有效值有关，和充电波形无关。电压的有效值用COM/SEG 之间的电压差值的均方根 V(RMS) 表示：

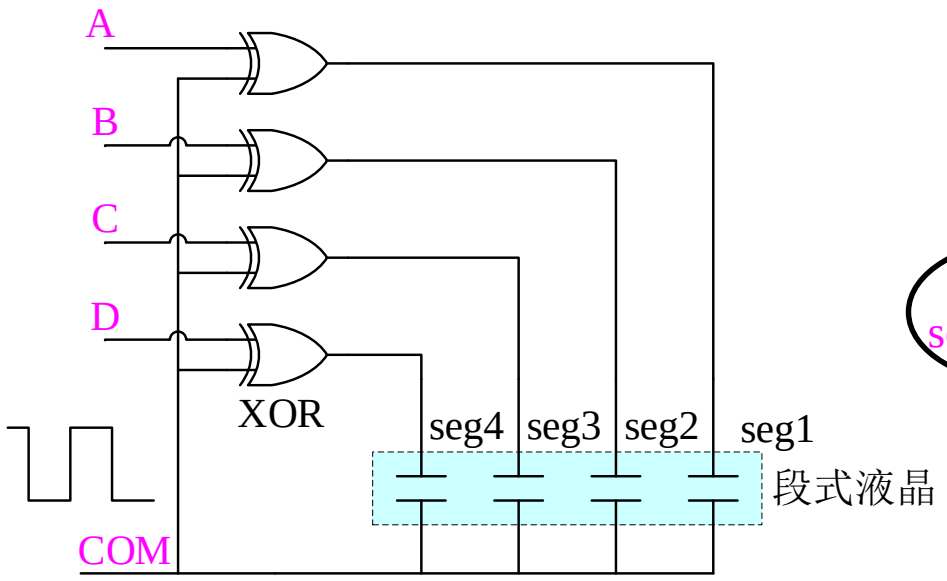
$$V(RMS) = \sqrt{\frac{1}{T}\int_0^T [V(t)]^2 dt}$$

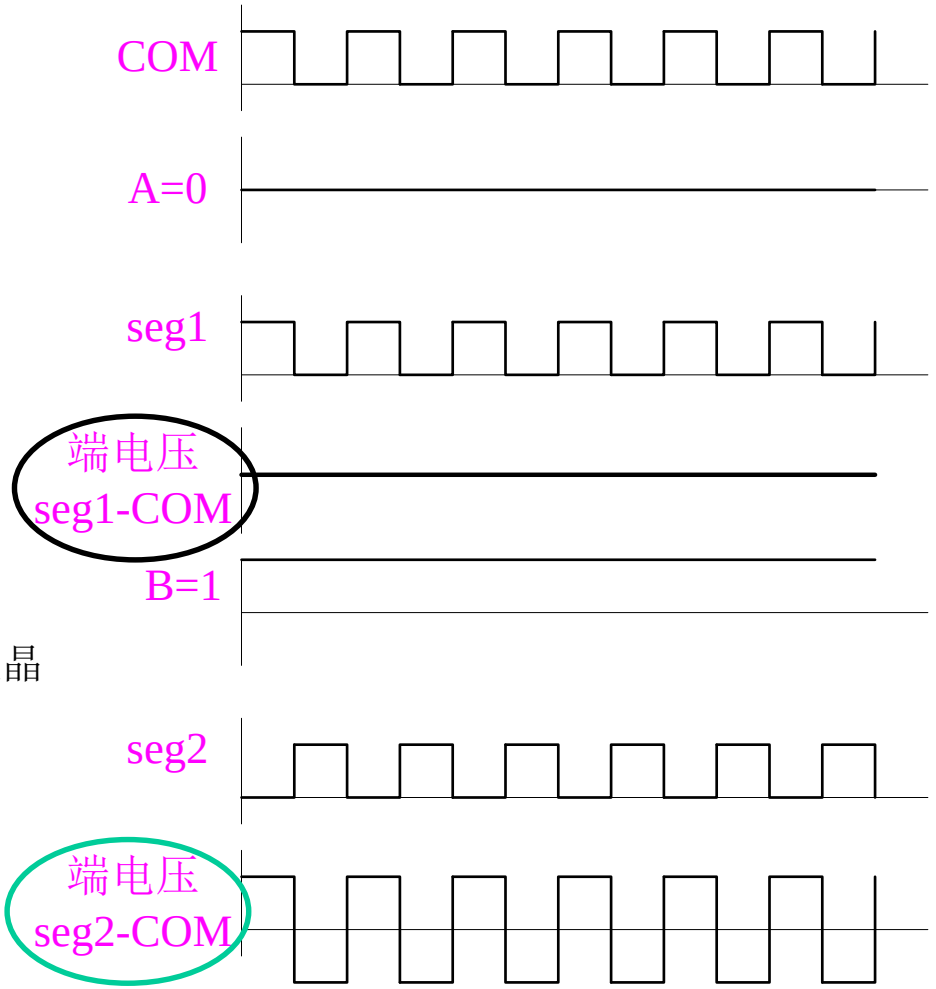
液晶驱动电压要求

液晶的透射率与交流电压有效值的关系：

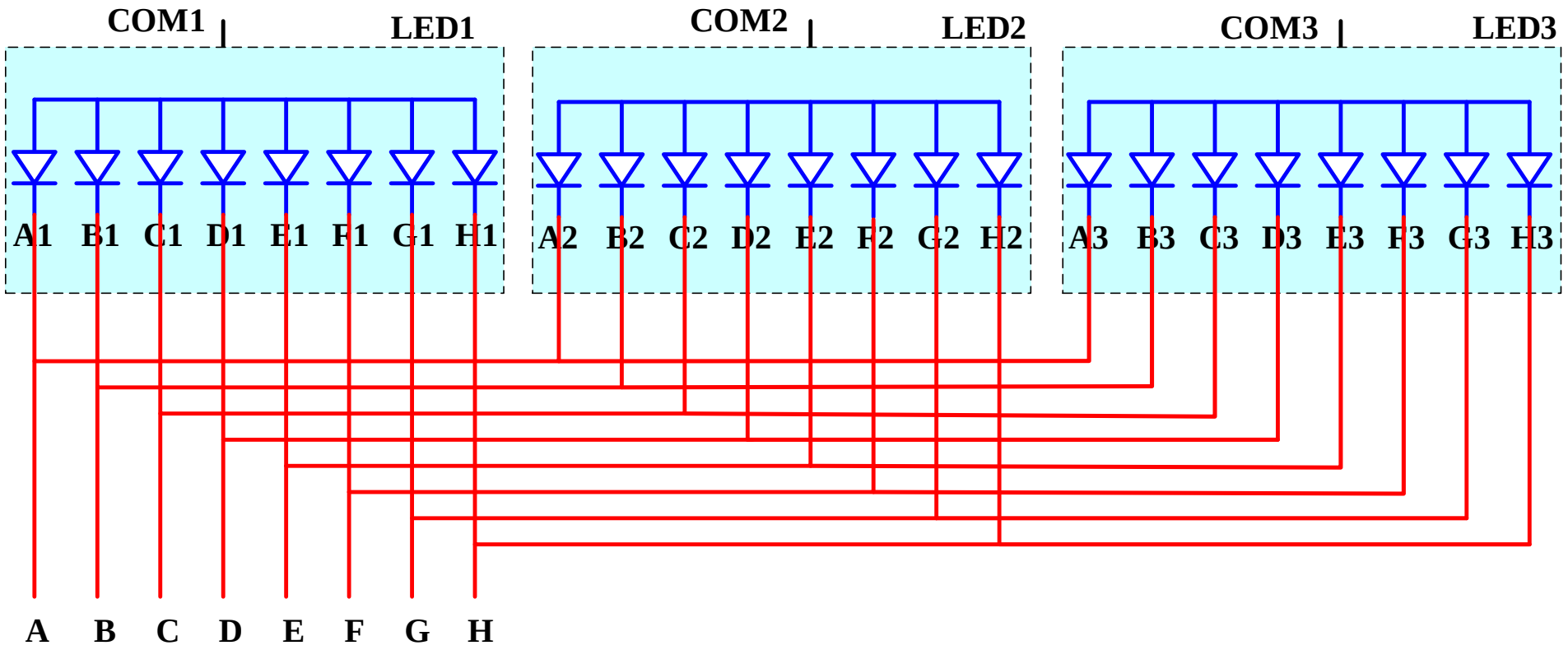


液晶驱动电路

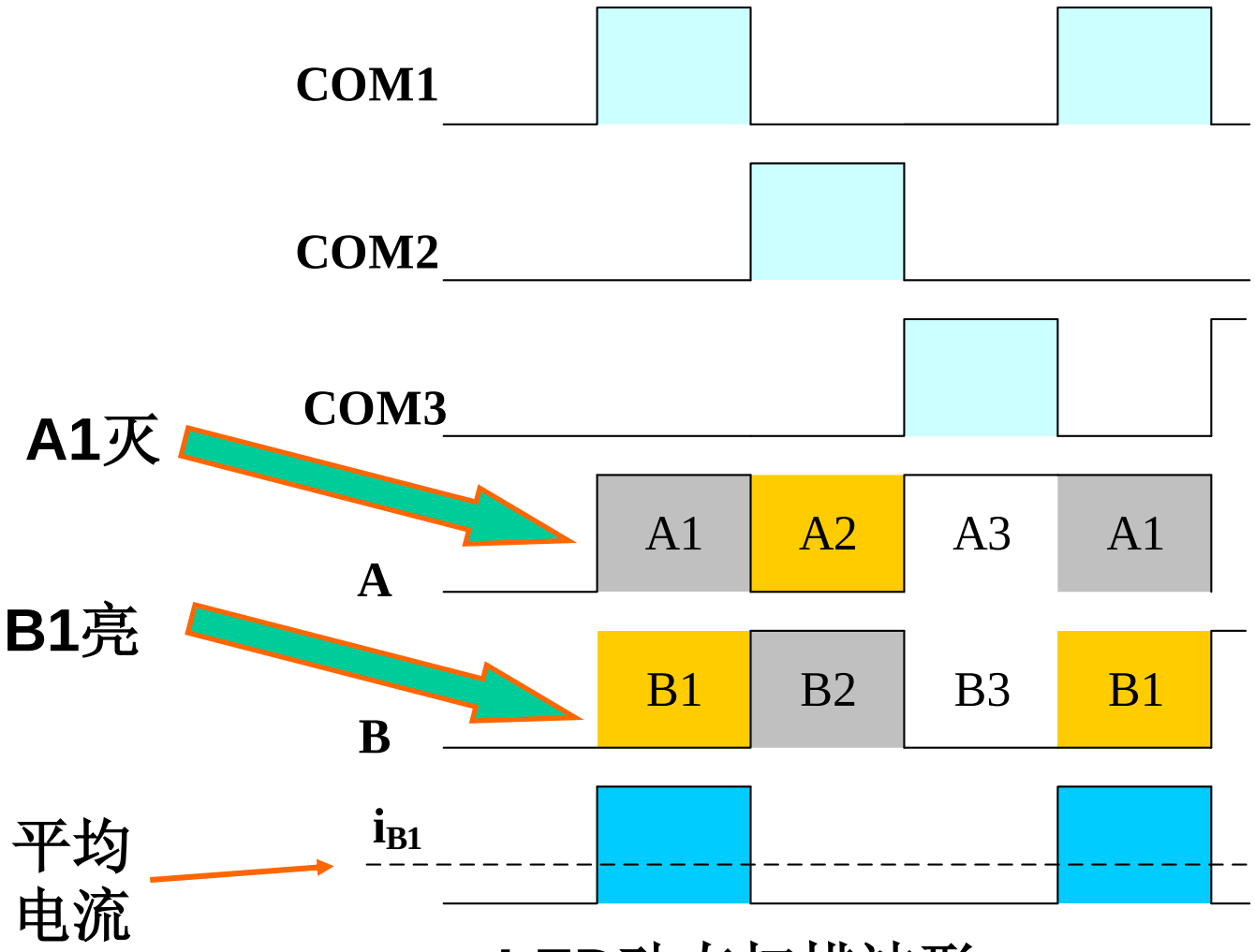

A
B
C
D
XOR
COM
seg4
seg3
seg2
seg1
段式液晶


COM
A=0
seg1
端电压
seg1-COM
B=1
seg2
端电压
seg2-COM

数码管动态显示

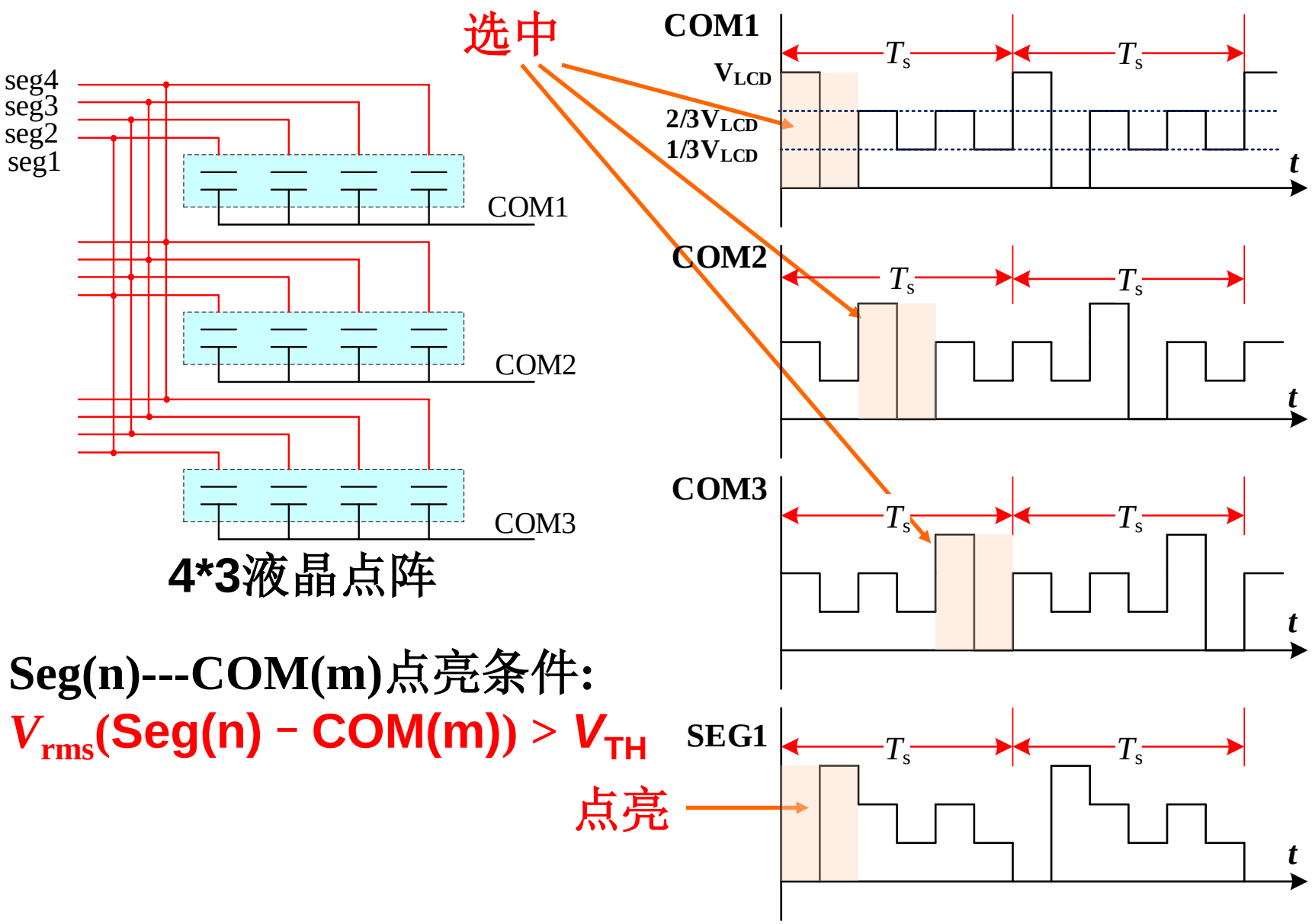

COM1
LED1
COM2
LED2
COM3
LED3
A1 B1 C1 D1 E1 F1 G1 H1
A2 B2 C2 D2 E2 F2 G2 H2
A3 B3 C3 D3 E3 F3 G3 H3
A B C D E F G H

LED数码管动态显示



LED动态扫描波形

液晶点阵驱动电路



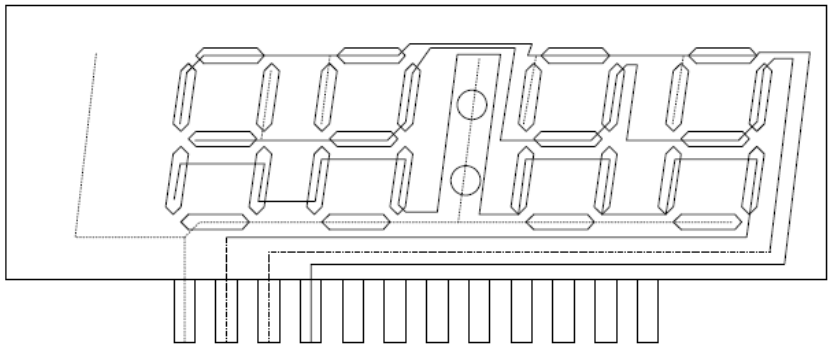
4*3液晶点阵

Seg(n)---COM(m)点亮条件:

$V_{rms}(Seg(n) - COM(m)) > V_{TH}$

点亮

LCD扫描显示波形很复杂



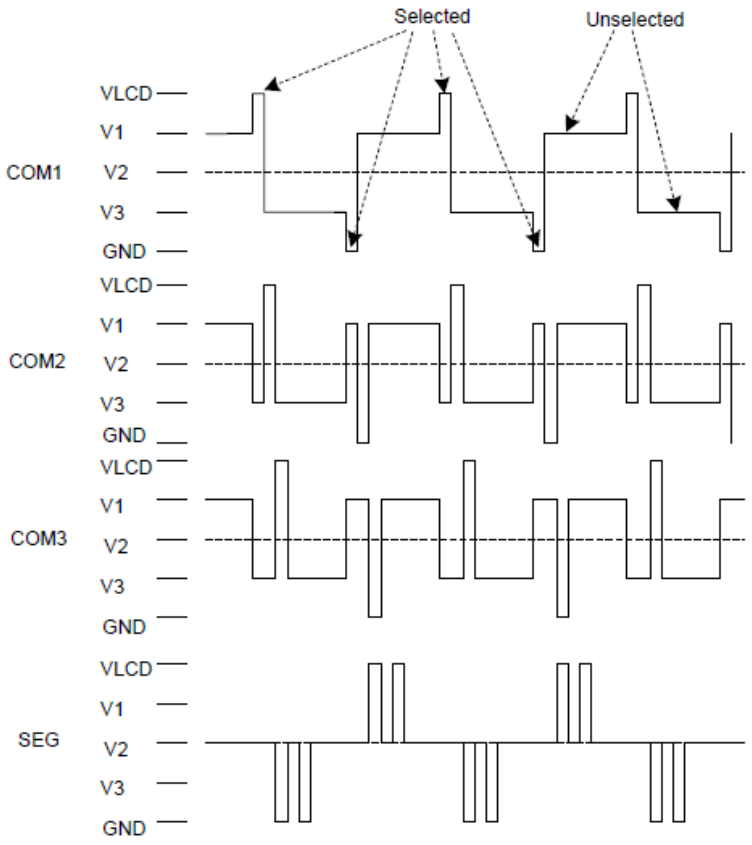
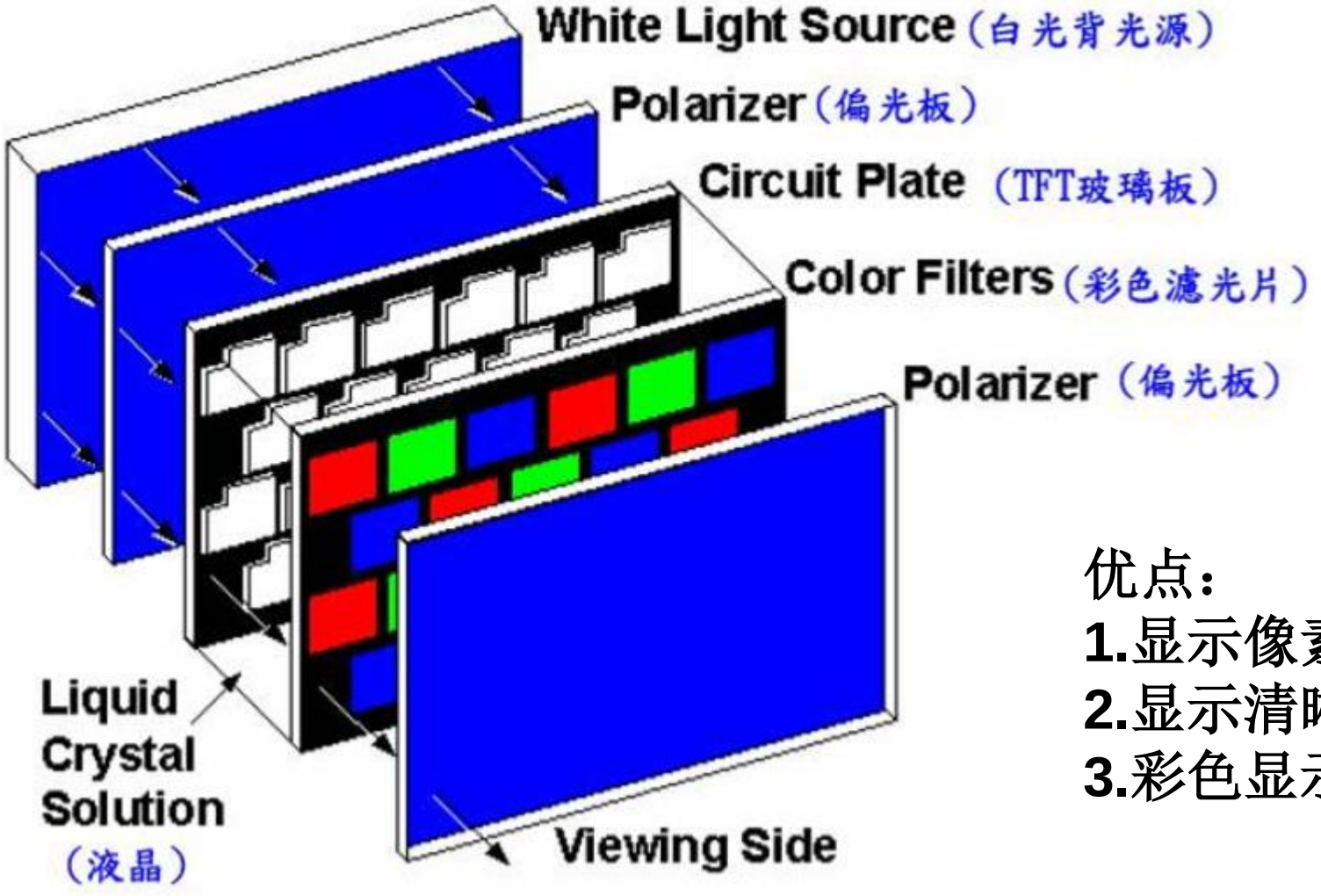


图 1-10 SEG/COM 波形图

TFT-LCD液晶原理

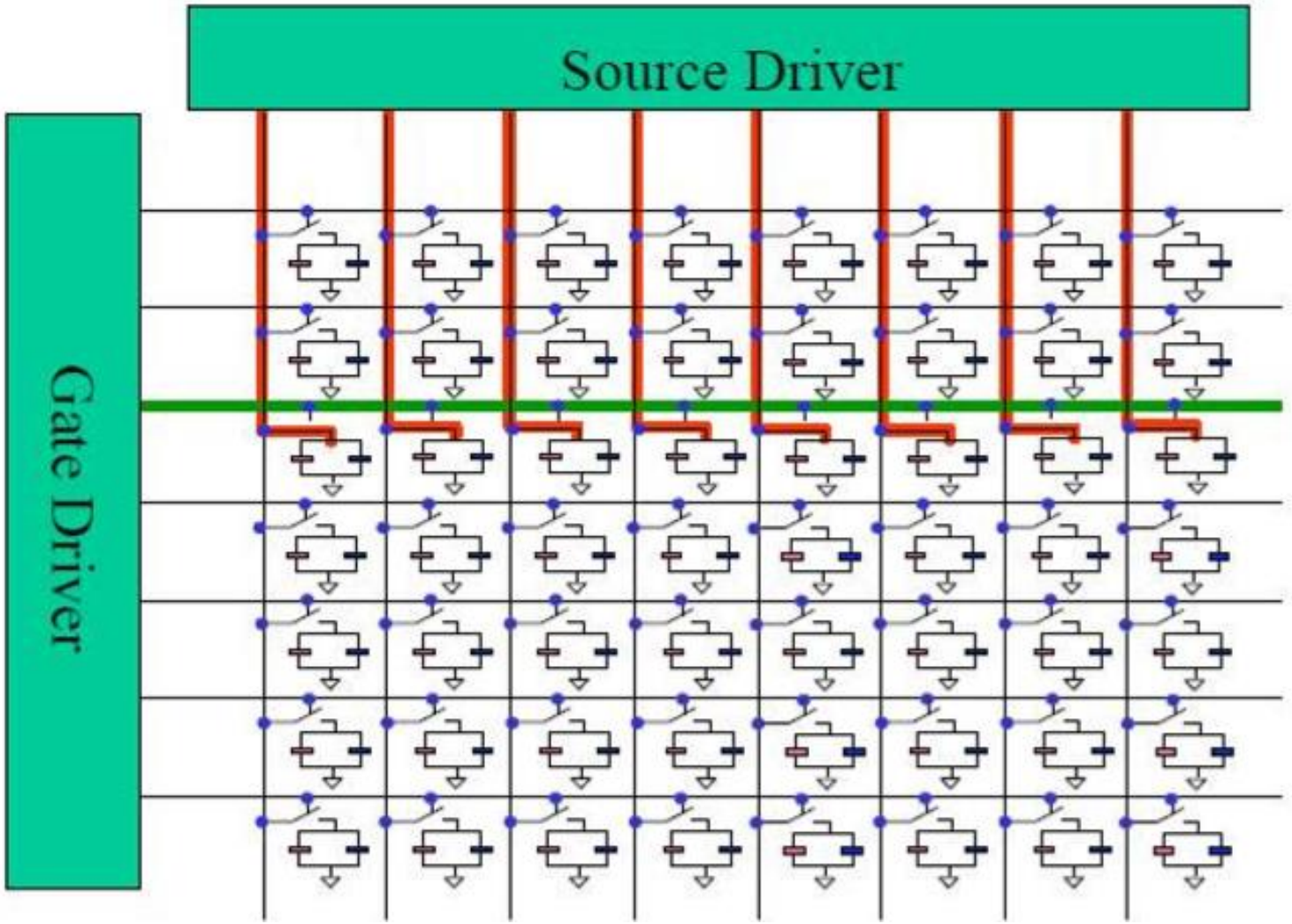
TFT（Thin Film Transistor）：薄膜场效应晶体管
在玻璃板上集成场效应晶体管对每个像素进行独立控制。



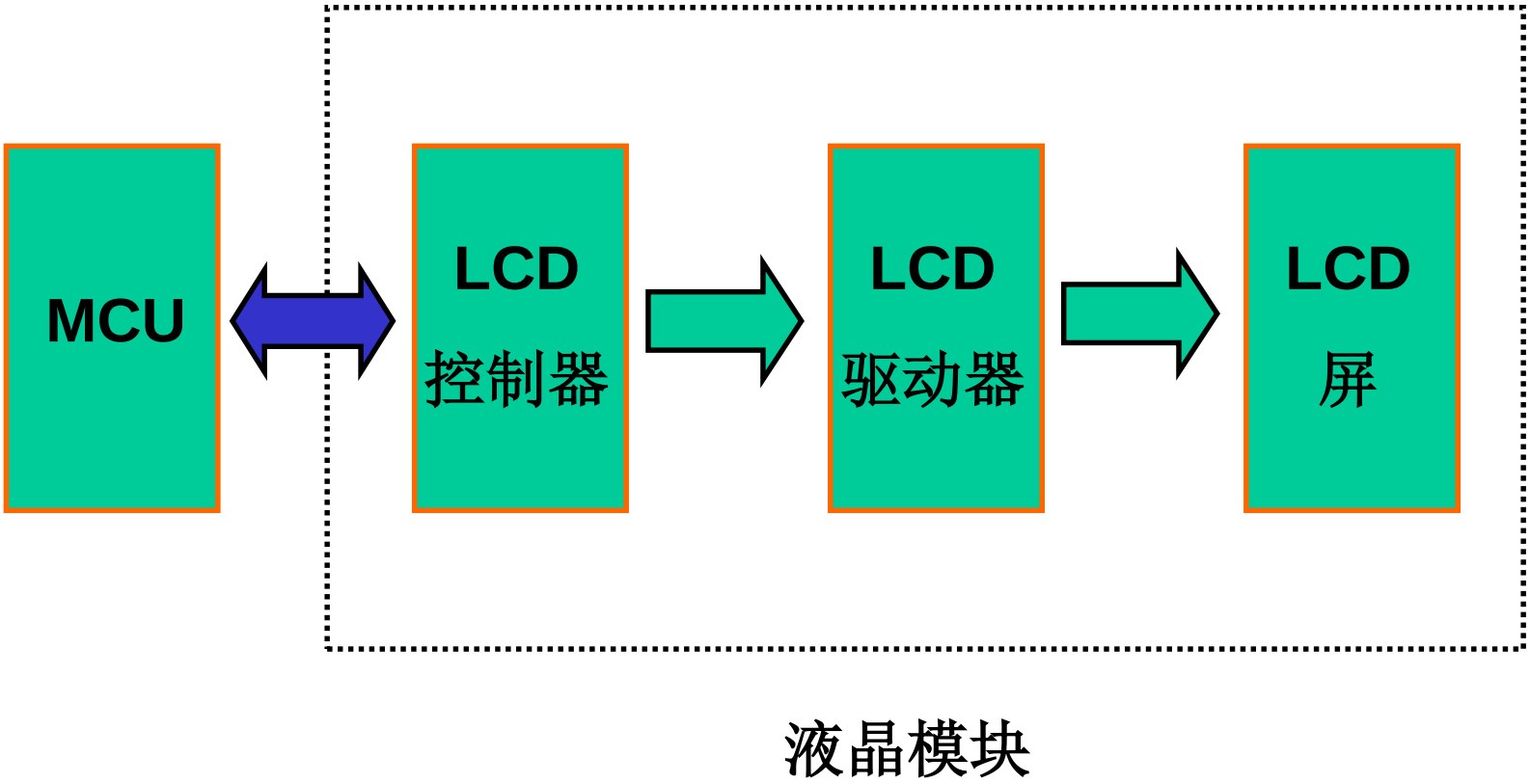
优点：
1.显示像素多；
2.显示清晰；
3.彩色显示

TFT-LCD液晶原理

控制信号



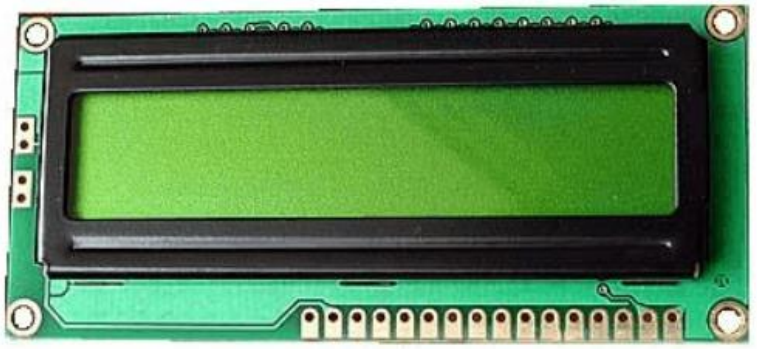
液晶显示电路组成



字符型和点阵型液晶模块

- **字符型：**

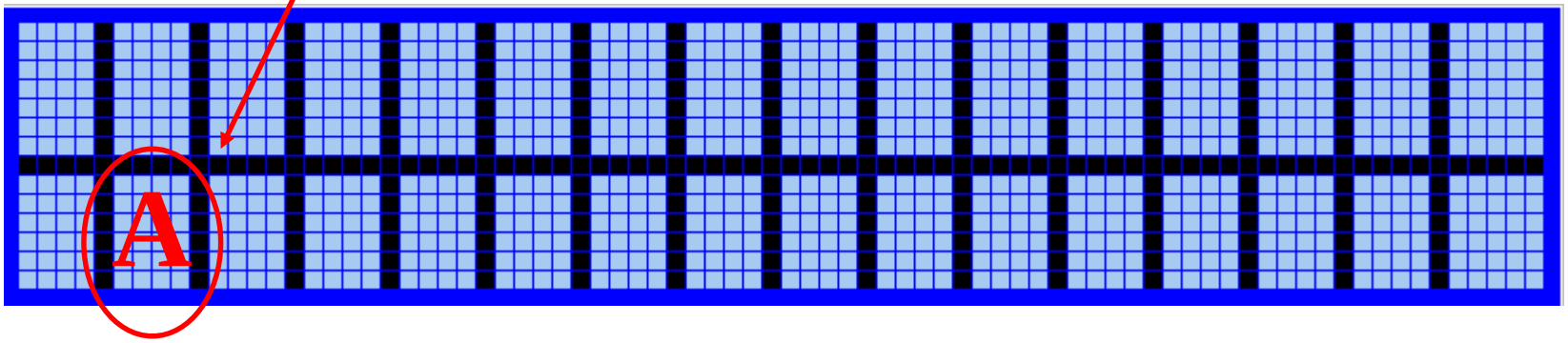
内部存有点阵图形（如5*7，5*8点阵），显示字符只需在相应的显示缓存中设置字符代码（内码）。



优点：编程简单，执行速度快。

缺点：灵活性差

	1	2	3	4		16
第一行	20H	20H	20H	20H	……	20H
第二行	20H	41H	20H	20H	……	20H



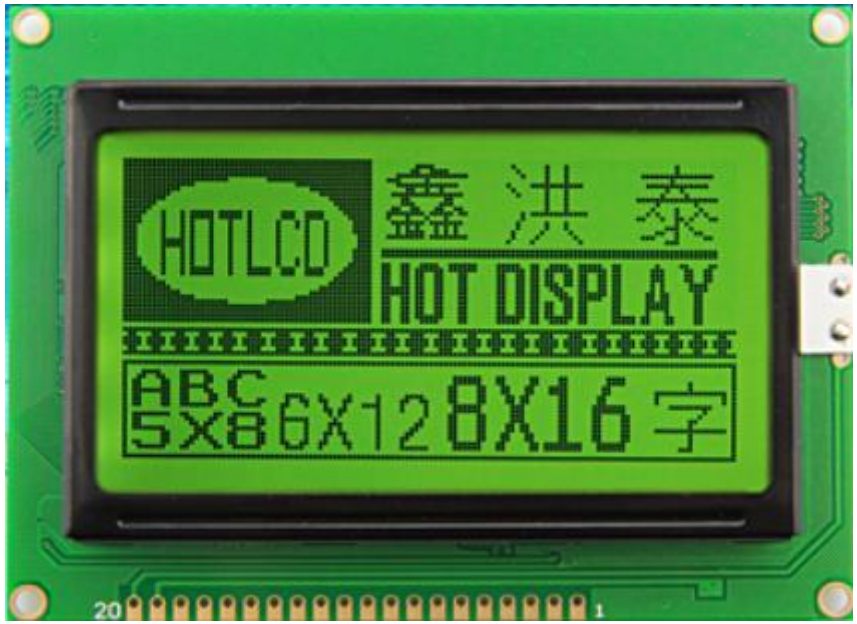
字符型和点阵型液晶模块

- **点阵型：**

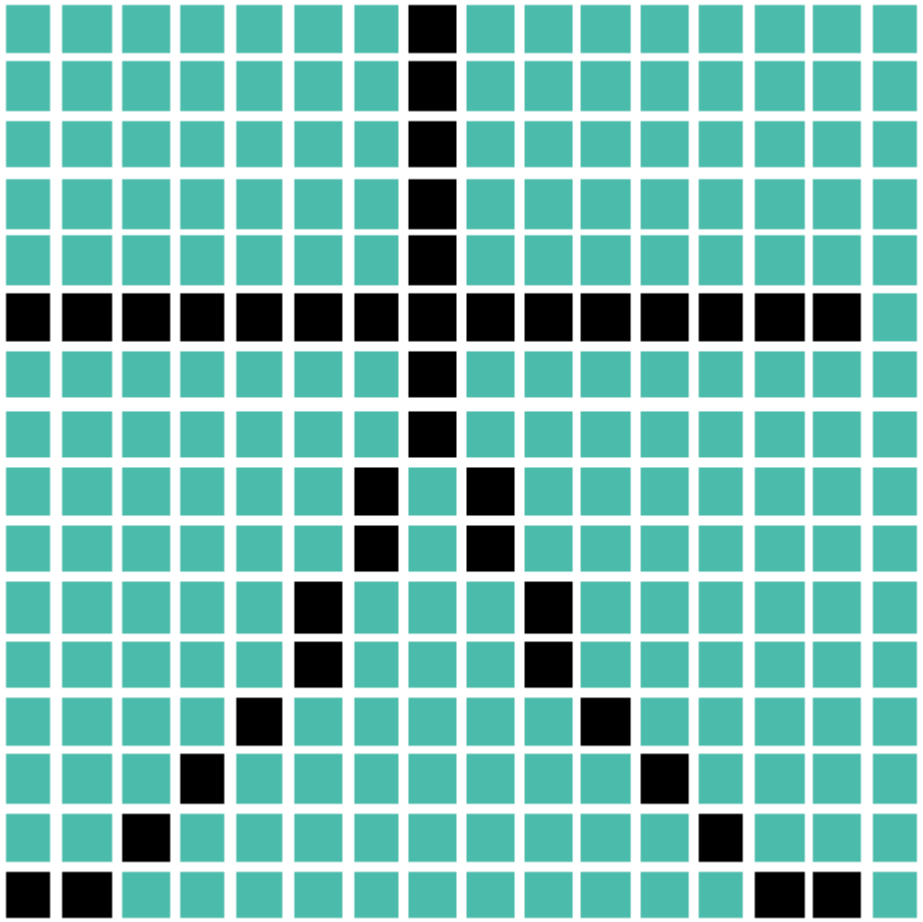
LCD显示屏的每个点对应显示RAM的一个bit，可显示字符或图案。需要在程序里设置字库。

优点：显示灵活性好；

缺点：编程复杂，数据量大。需要提取所有显示字符的字模。



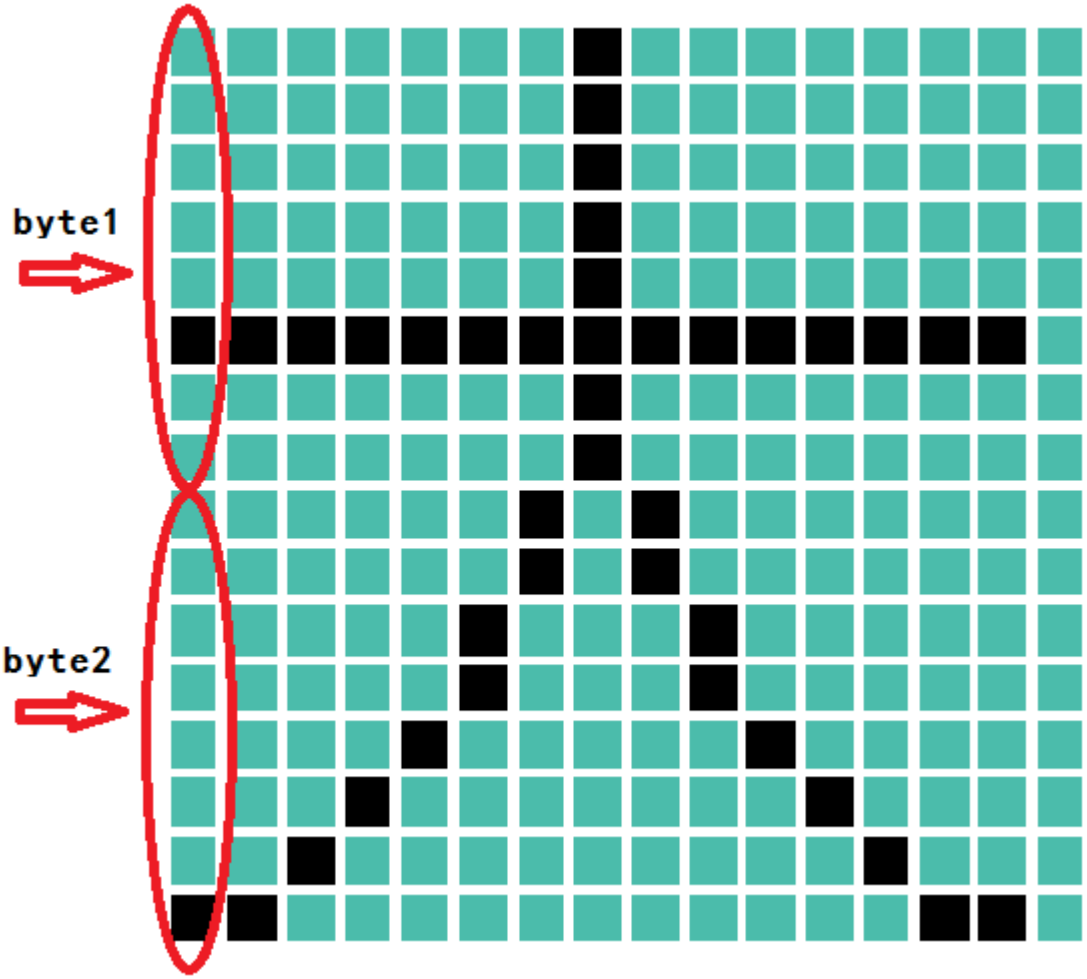
点阵字模提取



每一点对应**1bit**

16×16点阵对应
32 bytes

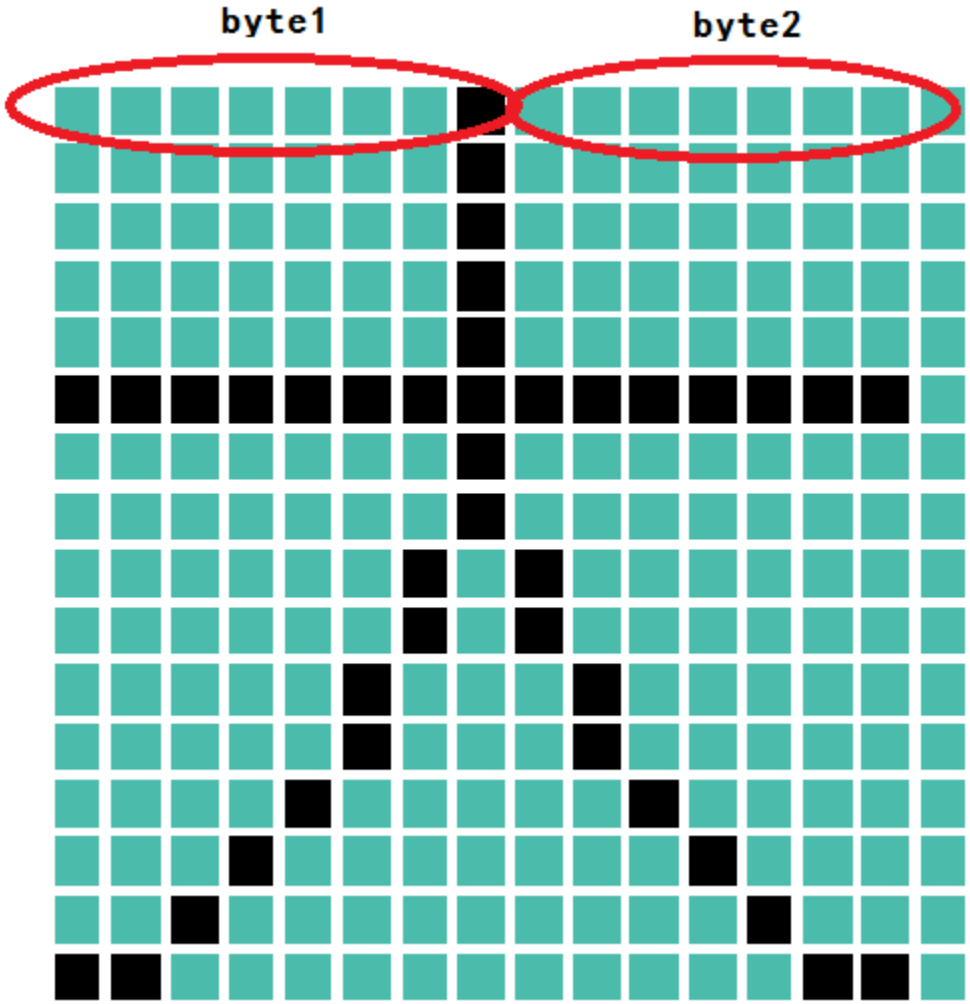
点阵字模提取



字模排列方式1：

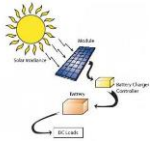
纵向排列

点阵字模提取



字模排列方式2：

横向排列



从字库中提取点阵字模

早期UCDOS的汉字字模保存在文件HZK16中（16*16）点阵。

汉字存放按区号和位号排列，每个区94个汉字。

汉字的顺序位置：（区号-1）*94+（位号-1）

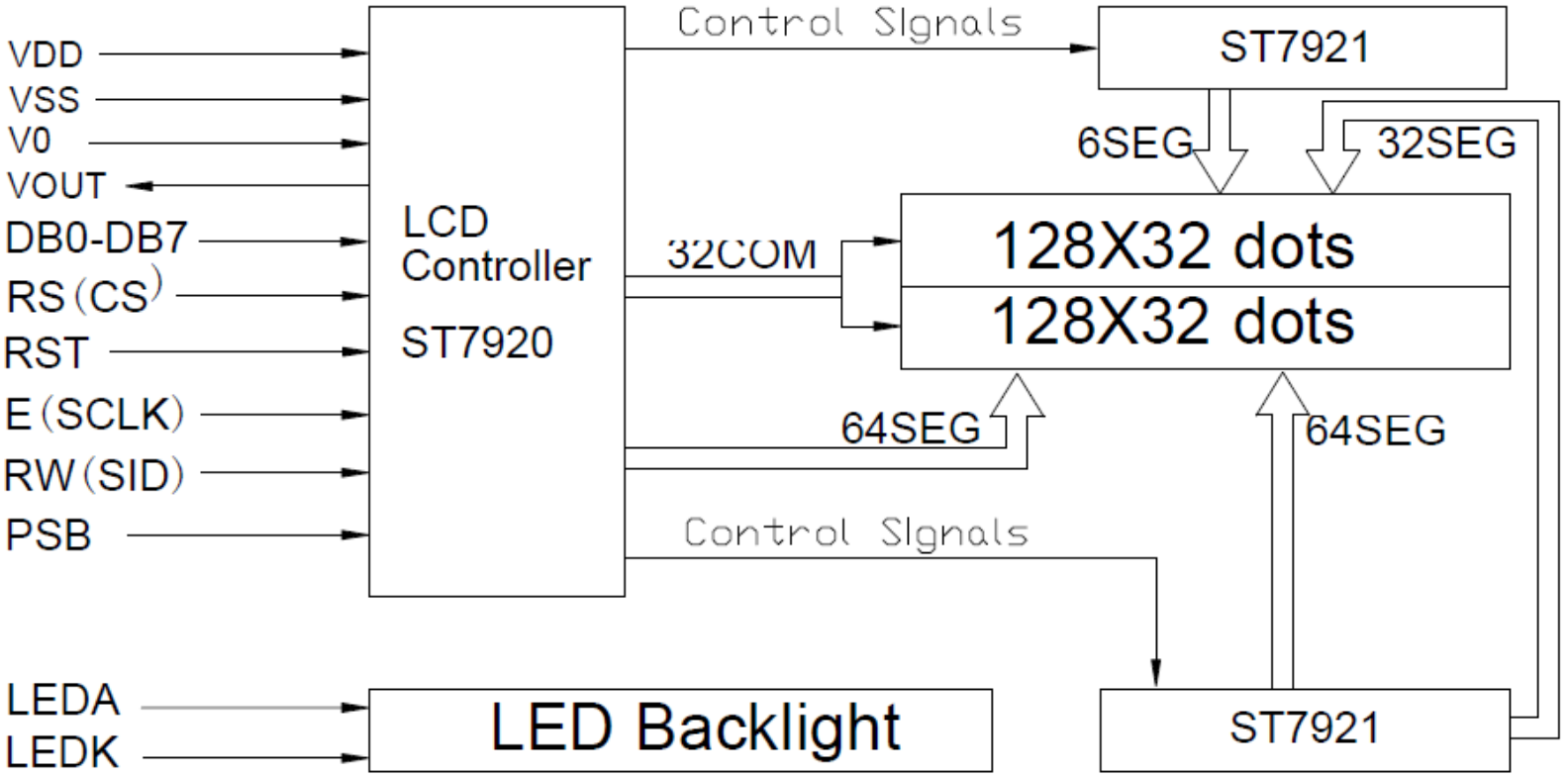
编程实现方法：

1. 读汉字“内码”（2字节），如调用ASC函数得到；

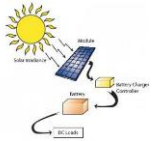
2.“大”字在文件中的位置：

[(内码1-161)*94+(内码2-161)]*32

液晶模块FYD12864原理图



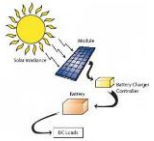
原理简图



液晶模块12864功能特性

基本特性:

- 低电源电压（VDD:+3.0--+5.5V）
- 显示分辨率:128×64 点
- 内置汉字字库，提供 8192 个 16×16 点阵汉字(简繁体可选)
- 内置 128 个 16×8 点阵字符
- 2MHZ 时钟频率
- 显示方式：STN、半透、正显
- 驱动方式：1/32DUTY，1/5BIAS
- 视角方向：6 点
- 背光方式：侧部高亮白色 LED，功耗仅为普通 LED 的 1/5—1/10
- 通讯方式：串行、并口可选
- 内置 DC-DC 转换电路，无需外加负压
- 无需片选信号，简化软件设计
- 工作温度：0℃ - +55℃ ，存储温度：-20℃ - +60℃

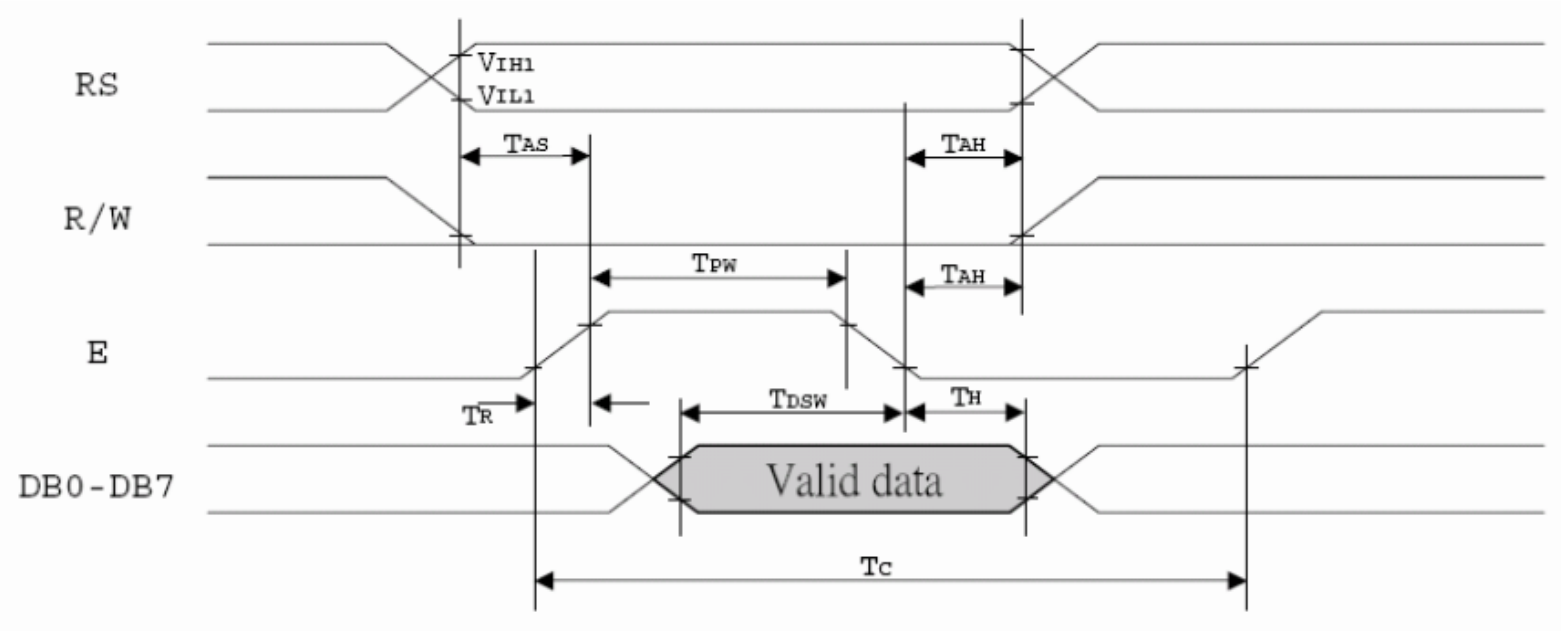


LCD点阵控制/驱动芯片ST7920

- ST7920 同时作为控制器和驱动器，它可提供 32 路 com 输出和 64 路 seg 输出。在驱动器 ST7921 的配合下，最多可以驱动 256×32 点阵液晶。

ST7920写操作时序

1. 8 位并口写操作时序图



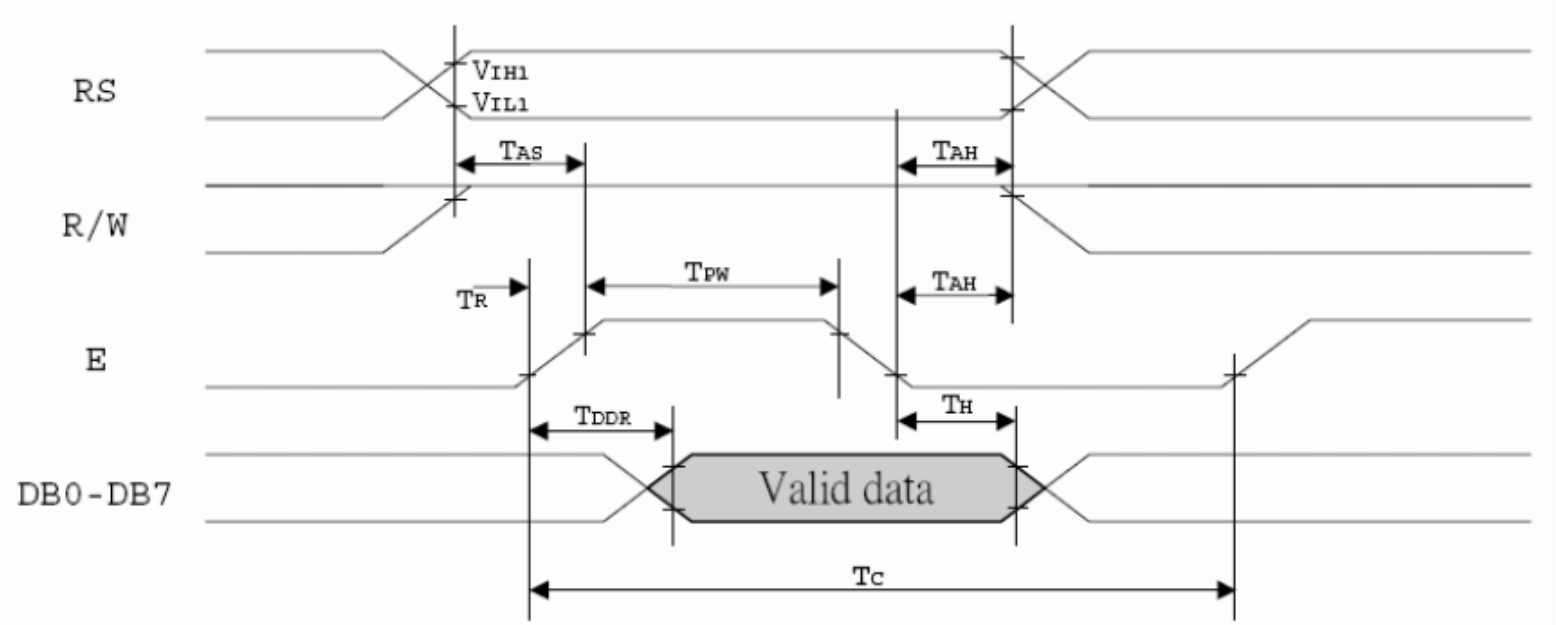
Motorala时序：读写时序由E，R/W 引脚实现；

Intel时序：/WR，/RD引脚控制读/写操作；

Intel时序与Motorala时序可以相互转化。

ST7920写操作时序

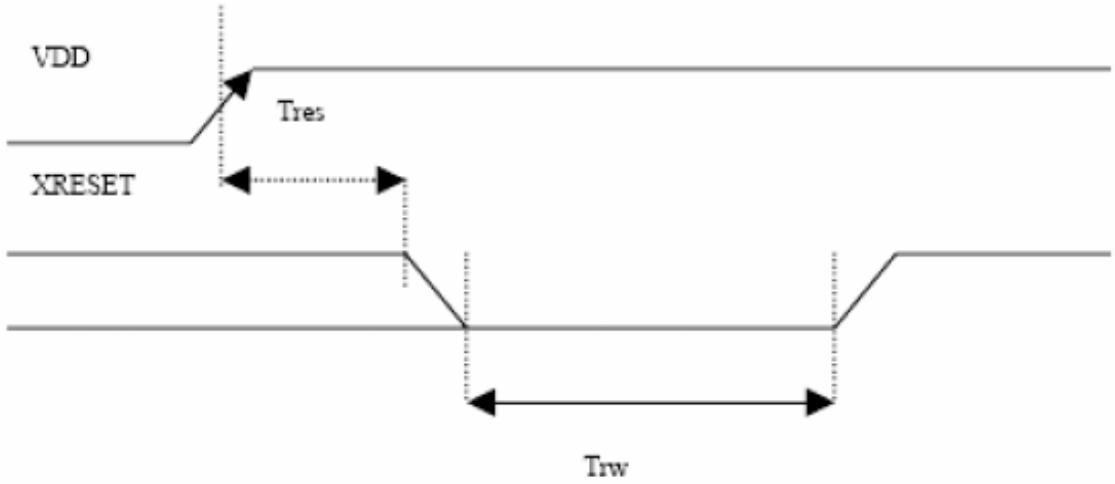
2. 8 位并口读操作时序图



RS	R/W	功能说明
L	L	MPU 写指令到指令暂存器（IR）
L	H	读出忙标志（BF）及地址记数器（AC）的状态
H	L	MPU 写入数据到数据暂存器（DR）

ST7920写操作时序

5. 外部复位时序图



XRESET pulse width	Trw	10us
RESET start time	Tres	50ns

引脚排列

管脚号	管脚名称	电平	管脚功能描述
1	VSS	0V	电源地
2	VCC	3.0+5V	电源正
3	V0	-	对比度（亮度）调整
4	RS(CS)	H/L	RS=“H”,表示 DB7——DB0 为显示数据 RS=“L”,表示 DB7——DB0 为显示指令数据
5	R/W(SID)	H/L	R/W=“H”,E=“H”,数据被读到 DB7——DB0 R/W=“L”,E=“H→L”， DB7——DB0 的数据被写到 IR 或 DR
6	E(SCLK)	H/L	使能信号
7～14	DB0 ～ DB7	H/L	三态数据线
15	PSB	H/L	H：8 位或 4 位并口方式，L：串口方式（见注释 1）
16	NC	-	空脚
17	/RESET	H/L	复位端，低电平有效（见注释 2）
18	VOUT	-	LCD 驱动电压输出端
19	A	VDD	背光源正端（+5V）（见注释 3）
20	K	VSS	背光源负端（见注释 3）

- **忙标志:BF**

BF 标志提供内部工作情况.BF=1 表示模块在进行内部操作,此时模块不接受外部指令和数据.BF=0 时,模块为准备状态,随时可接受外部指令和数据.

利用 STATUS RD 指令,可以将 BF 读到 DB7 总线,从而检验模块之工作状态.

- **字型产生 ROM（CGROM）**

字型产生 ROM（CGROM）提供 8192 个此触发器是用于模块屏幕显示开和关的控制。DFF=1 为开显示（DISPLAY ON),DDRAM 的内容就显示在屏幕上，DFF=0 为关显示（DISPLAY OFF)。

DFF 的状态是指令 DISPLAY ON/OFF 和 RST 信号控制的。

- **显示数据 RAM（DDRAM）**

模块内部显示数据 RAM 提供 64×2 个位元组的空间，最多可控制 4 行 16 字（64 个字）的中文字型显示，当写入显示数据 RAM 时，可分别显示 CGROM 与 CGRAM 的字型；此模块可显示三种字型，分别是半角英数字型(16*8)、CGRAM 字型及 CGROM 的中文字型，三种字型的选择，由在 DDRAM 中写入的编码选择，在 0000H—0006H 的编码中（其代码分别是 0000、0002、0004、0006 共 4 个）将选择 CGRAM 的自定义字型，02H—7FH 的编码中将选择半角英数字的字型，至于 A1 以上的编码将自动的结合下一个位元组，组成两个位元组的编码形成中文字型的编码 BIG5（A140—D75F），GB（A1A0-F7FFH)。

- **字型产生 RAM(CGRAM)**

字型产生 RAM 提供图象定义(造字)功能， 可以提供四组 16×16 点的自定义图象空间,使用者可以将内部字型没有提供的图象字型自行定义到 CGRAM 中,便可和 CGROM 中的定义一样地通过 DDRAM 显示在屏幕中。

- **地址计数器 AC**

地址计数器是用来贮存 DDRAM/CGRAM 之一的地址,它可由设定指令暂存器来改变，之后只要读取或是写入 DDRAM/CGRAM 的值时，地址计数器的值就会自动加一，当 RS 为“0”时而 R/W 为“1”时，地址计数器的值会被读取到 DB6——DB0 中。

- **光标/闪烁控制电路**

此模块提供硬体光标及闪烁控制电路，由地址计数器的值来指定 DDRAM 中的光标或闪烁位置。

指令表

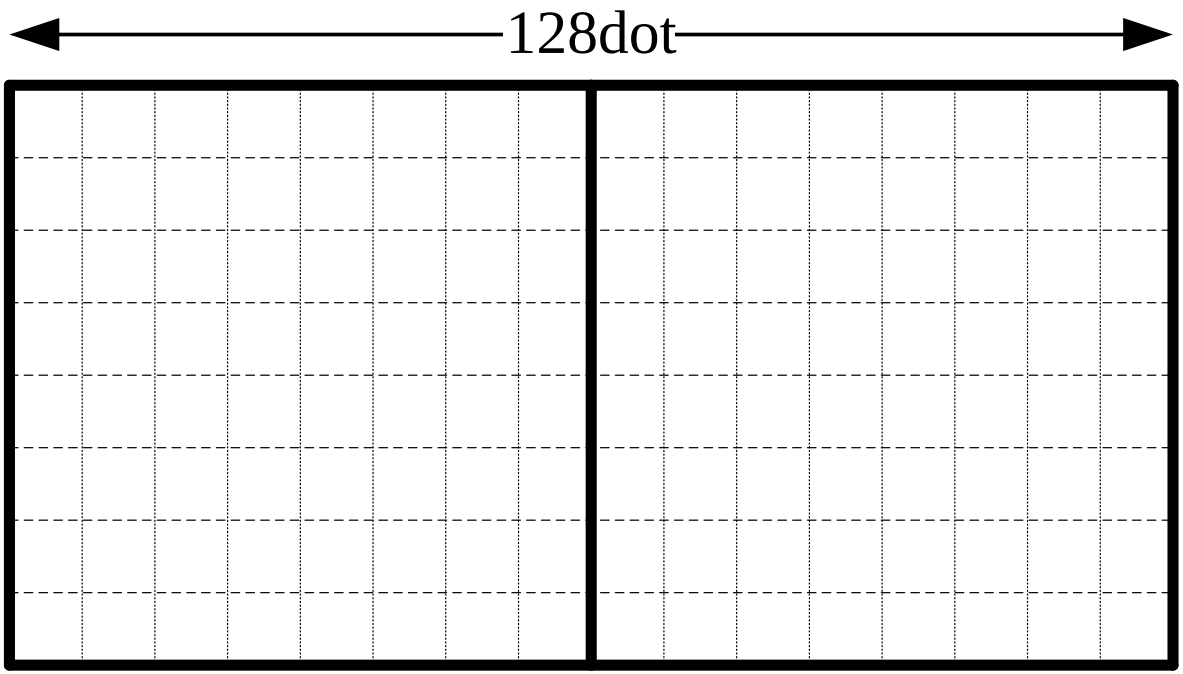
指令表 1：（RE=0：基本指令）

指令	指令码										功能
	RS	R/W	D7	D6	D5	D4	D3	D2	D1	D0	
清除显示	0	0	0	0	0	0	0	0	0	1	将DDRAM填满"20H"，并且设定DDRAM的地址计数器(AC)到"00H"
地址归位	0	0	0	0	0	0	0	0	1	X	设定 DDRAM 的地址计数器(AC)到"00H"，并且将游标移到开头原点位置；这个指令不改变 DDRAM 的内容
显示状态开/关	0	0	0	0	0	0	1	D	C	B	D=1：整体显示 ON C=1：游标 ON B=1：游标位置反白允许
进入点设定	0	0	0	0	0	0	0	1	I/D	S	指定在数据的读取与写入时，设定游标的移动方向及指定显示的移位
游标或显示移位控制	0	0	0	0	0	1	S/C	R/L	X	X	设定游标的移动与显示的移位控制位；这个指令不改变 DDRAM 的内容
功能设定	0	0	0	0	1	DL	X	RE	X	X	DL=0/1：4/8 位数据 RE=1：扩充指令操作 RE=0：基本指令操作

指令表

设定 CGRAM 地址	0	0	0	1	AC5	AC4	AC3	AC2	AC1	AC0	设定 CGRAM 地址
设定 DDRAM 地址	0	0	1	0	AC5	AC4	AC3	AC2	AC1	AC0	设定 DDRAM 地址（显示位址） 第一行：80H－87H 第二行：90H－97H
读取忙标志和地址	0	1	BF	AC6	AC5	AC4	AC3	AC2	AC1	AC0	读取忙标志(BF)可以确认内部动作是否完成，同时可以读出地址计数器(AC)的值
写数据到 RAM	1	0	数据								将数据 D7——D0 写入到内部的 RAM (DDRAM/CGRAM/IRAM/GRAM)
读出 RAM 的值	1	1	数据								从内部 RAM 读取数据 D7——D0 (DDRAM/CGRAM/IRAM/GRAM)

字符方式显示



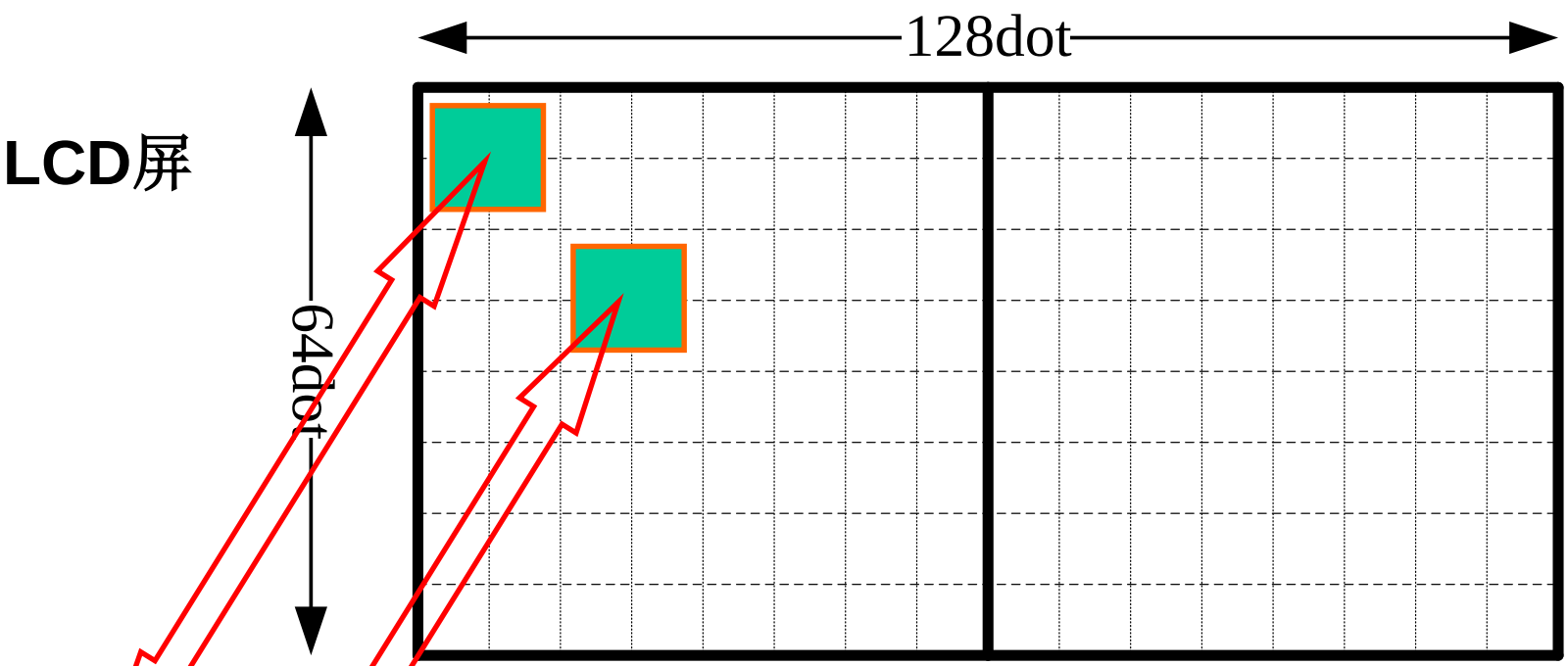
LCD屏

DDRAM区

80H	81H	82H	83H	84H	85H	86H	87H
90H	91H	92H	93H	94H	95H	96H	97H
88H	89H	8AH	8BH	8CH	8DH	8EH	8FH
98H	99H	9AH	9BH	9CH	9DH	9EH	9FH

汉字显示区域与字符显示RAM对应表

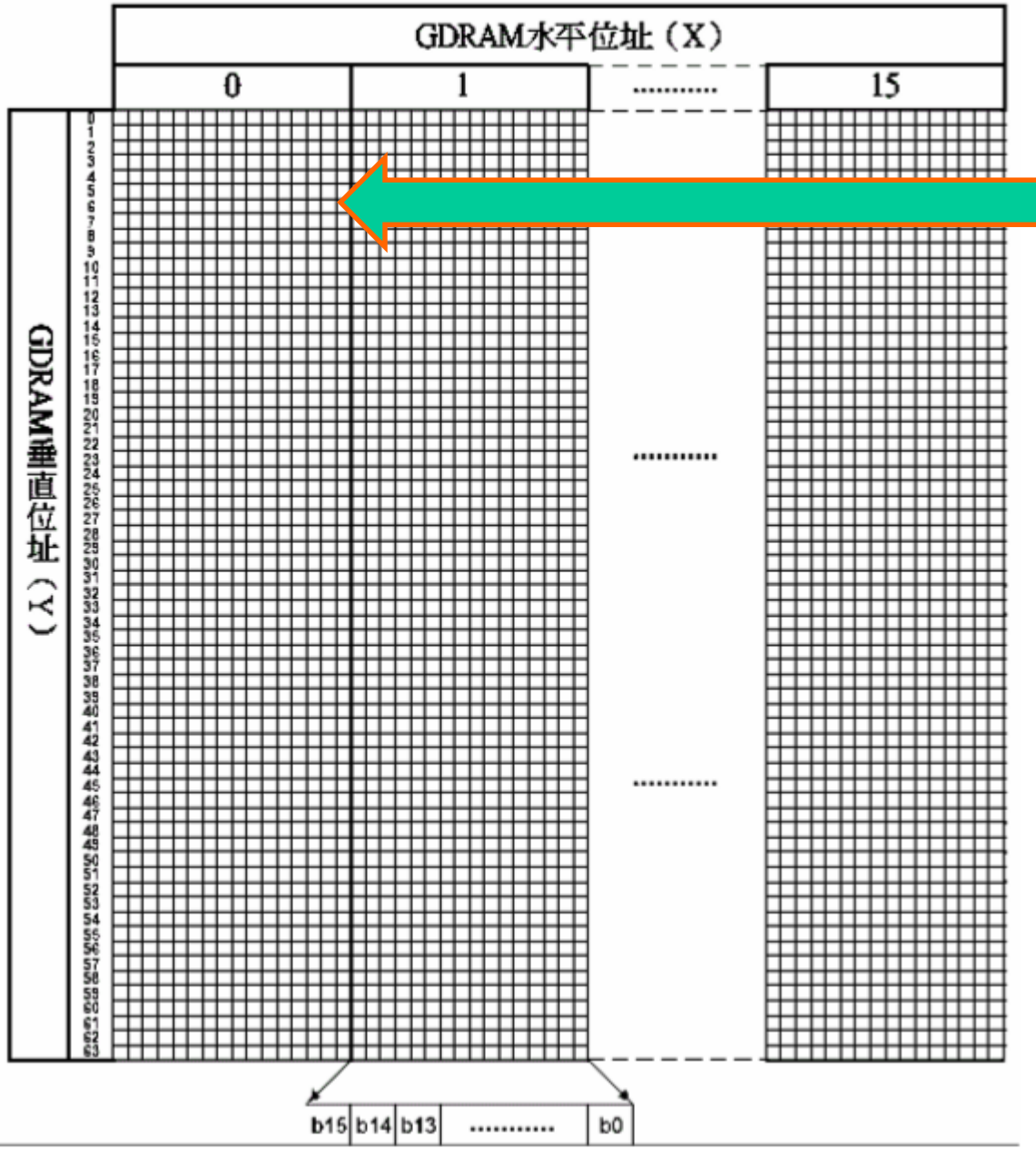
字符方式显示

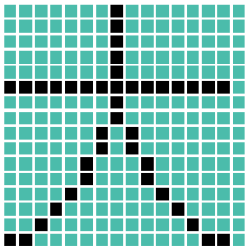


80H	81H	82H	83H	84H	85H	86H	87H
90H	91H	92H	93H	94H	95H	96H	97H
88H	89H	8AH	8BH	8CH	8DH	8EH	8FH
98H	99H	9AH	9BH	9CH	9DH	9EH	9FH

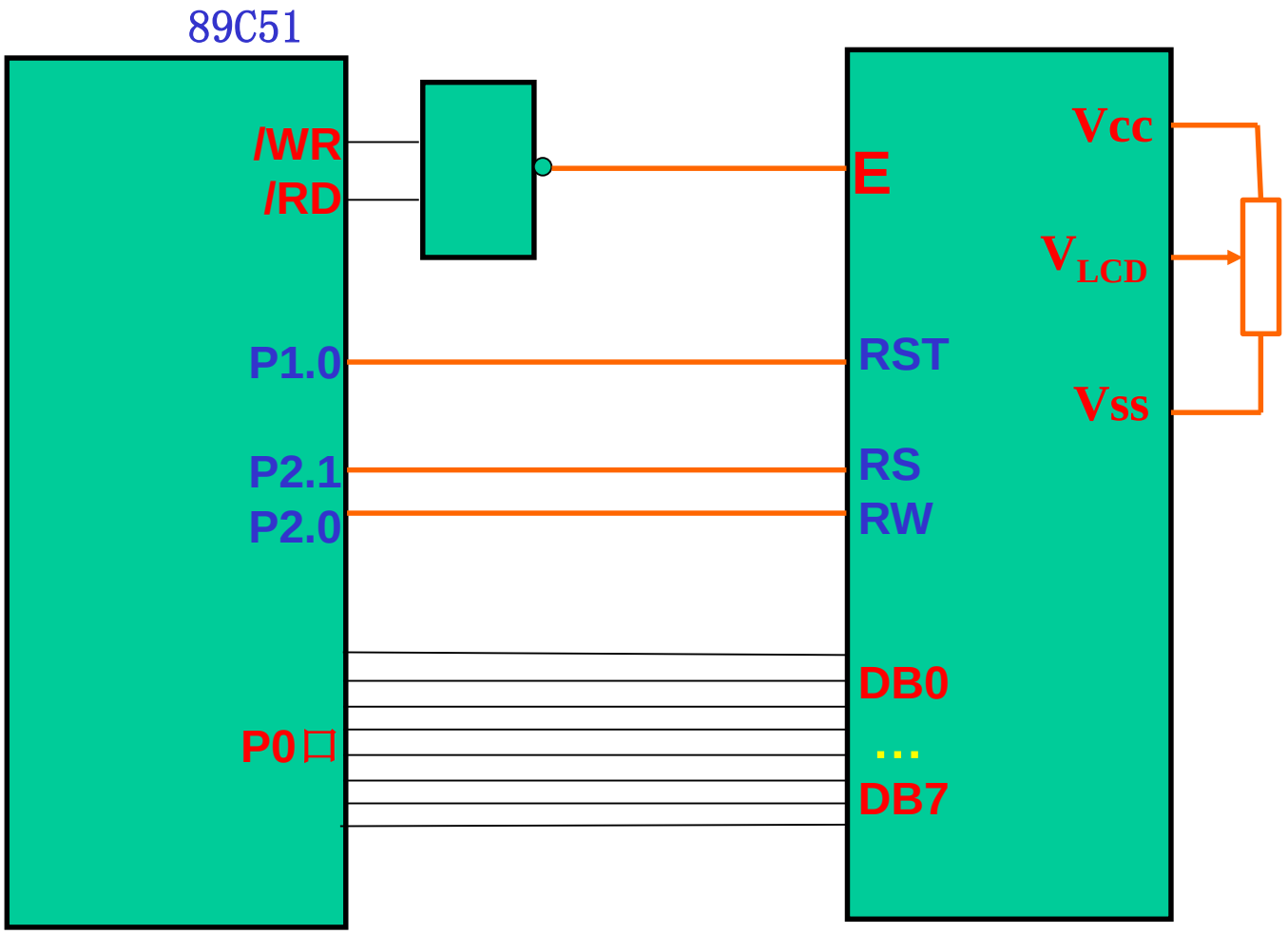
汉字显示区域与字符显示RAM对应表

图形方式显示



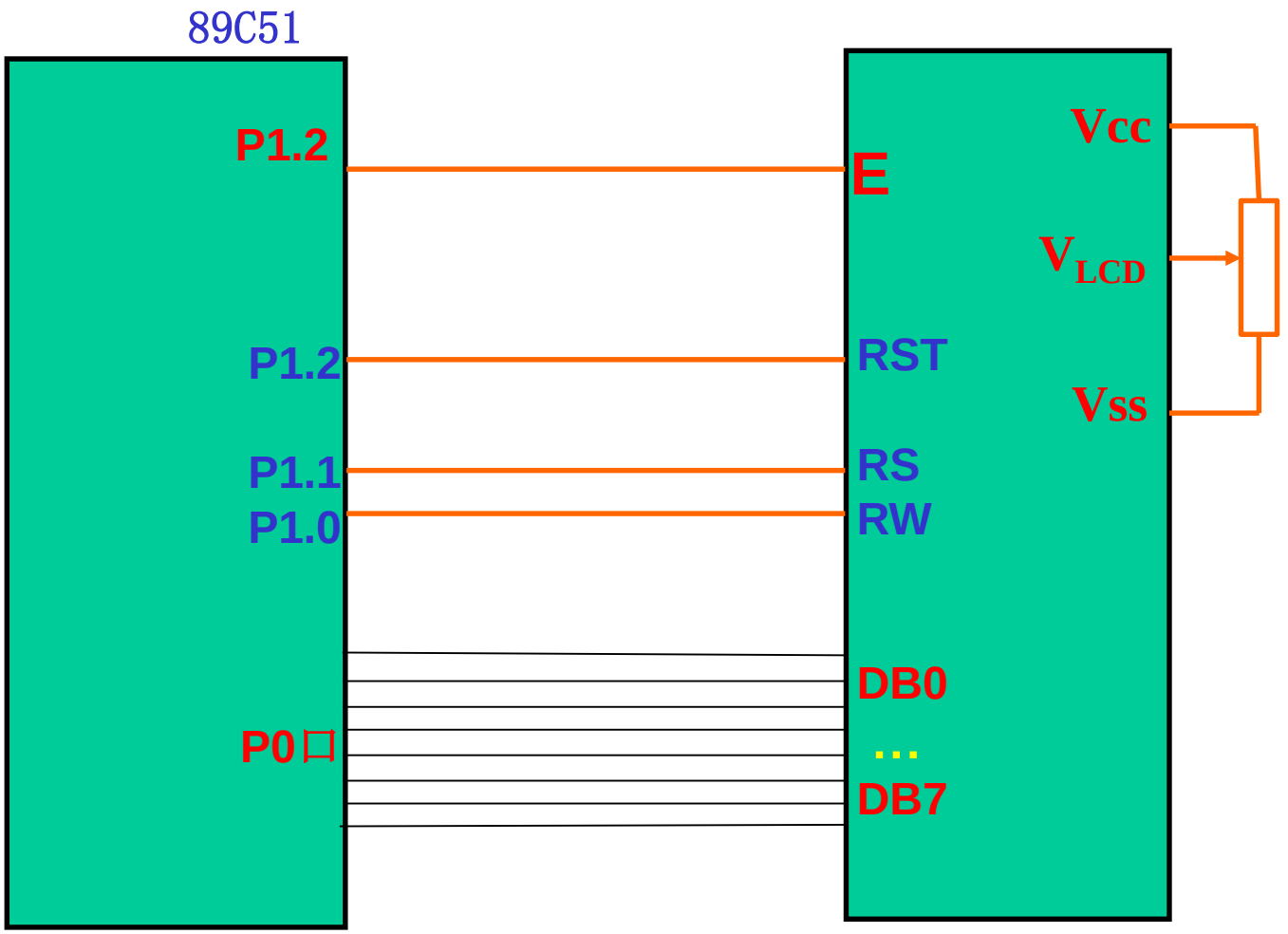


89C52与液晶模块接口



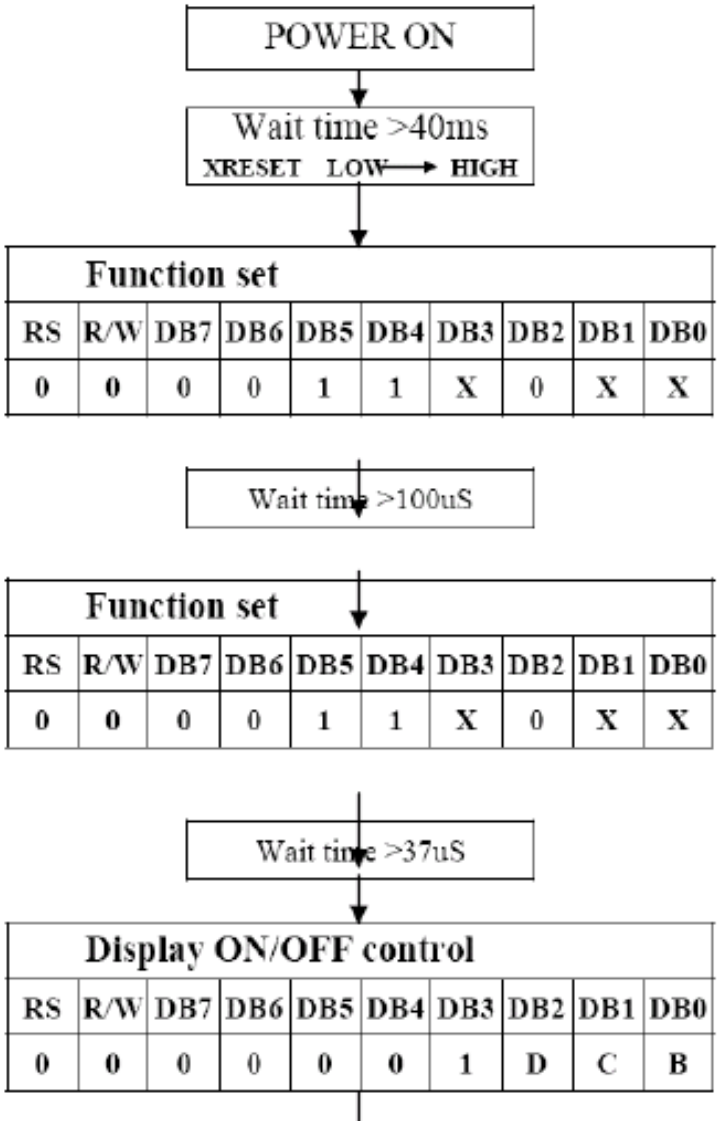
总线方式

89C52与液晶模块接口



非总线方式

初始化设计

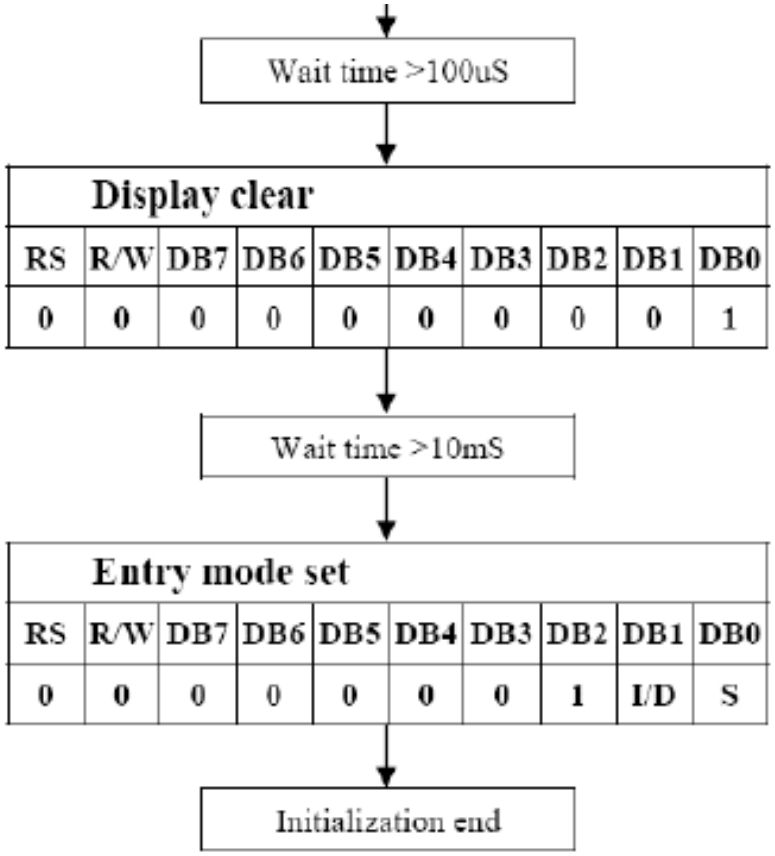



```
MOV     SP,#60H
CLR     RST
LCALL   DELAY40MS
SETB    RST
NOP
SETB    PSB             ;***8位数据并口
**初始化*******
MOV     A,#30H      ;30H—基本指令操作
LCALL SEND_I

MOV     A,#30H      ;30H--基本指令操作
LCALL SEND_I

MOV     A,#0CH      ;开显示,关光标,不闪烁
LCALL SEND_I
```

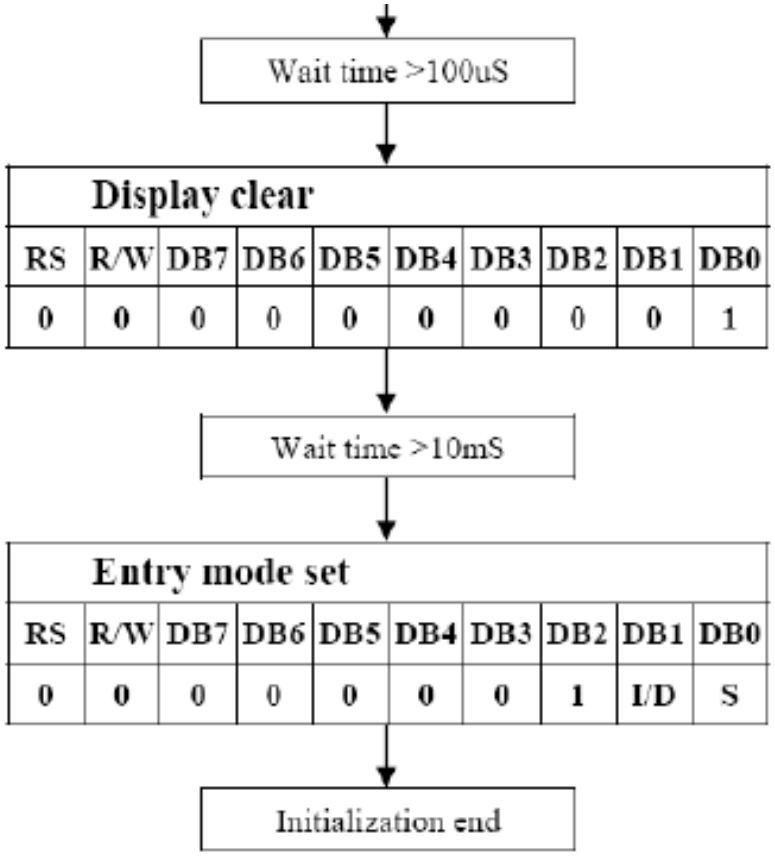
初始化设计




```
MOV     A,#01H              ;清除显示
LCALL SEND_I
```

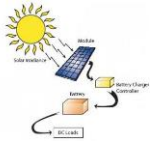
```
MOV     A,#06H              ;指定在资料写入或读取时，光标的移动方向
LCALL SEND_I        ;DDRAM 的地址计数器(AC)加1
```

初始化设计




```
MOV     A,#01H              ;清除显示
LCALL SEND_I
```

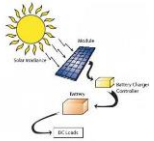
```
MOV     A,#06H              ;指定在资料写入或读取时，光标的移动方向
LCALL SEND_I        ;DDRAM 的地址计数器(AC)加1
```



汉字字符显示

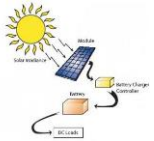
```
LCALL   CLEAR_P          ;;清屏
MOV     DPTR,#TABZ       ;;显示汉字和字符
MOV     COUNT,#10H       ;显示8个字符
MOV     A,#80H           ;起始地址
LCALL   SEND_I
LCALL   DISP_HZ
SJMP    $

TABZ:    DB '   浙江大学2013   '
```



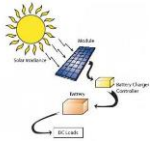
写命令子程序

```
;写指令子程序
;RS=0,RW=0,E=高脉冲,D0-D7=指令码
;*****************************************************
SEND_I:
        LCALL CHK_BUSY
        CLR     RS
        CLR     RW
        MOV     P0,A
        SETB E
        NOP
        NOP
        CLR     E
        RET
```



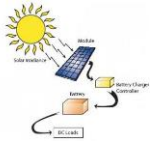
写数据子程序

```
; ;写数据子程序
;RS=1,RW=0,E=高脉冲,D0-D7=数据
;*****************************************************
SEND_D:
        LCALL   CHK_BUSY
        SETB    RS
        CLR     RW
        MOV     P0,A
        SETB    E
        NOP
        NOP
        CLR     E
        RET
```



测试忙子程序

```
;测忙子程序
;RS=0,RW=1,E=H,D0-D7=状态字
;*************************************************************
CHK_BUSY:
        MOV         P0,#0FFH    ;测忙碌子程序
        CLR         RS
        SETB        RW
        SETB        E
        JB  P0.7,$
        CLR         E
        RET
```



END

THANK YOU!